

sqtseries

Complete Codebase Guide — every file, class, function, constant and example explained

/home/iam/devcode/PiCCL/sqtseries

version 0.1.0

Python ≥ 3.14 + SQLite + ZeroMQ + FastAPI

~19k lines: 6.1k source · 4.2k examples · 8.5k tests

Generated 2026-08-10 · self-contained HTML, printable to PDF

`google-chrome --headless=new --disable-gpu --no-sandbox --print-to-pdf=codebase-guide.pdf codebase-guide.html`

Table of Contents

1. Project Overview

2. Architecture at a Glance

3. Core Concepts

4. Root Files

- 4.1 pyproject.toml
- 4.2 requirements.txt
- 4.3 README.md
- 4.4 .gitignore

5. Package sqtseries (top level)

- 5.1 __init__.py
- 5.2 __main__.py
- 5.3 client.py — Client, ClientError
- 5.4 cli.py — CLI commands
- 5.5 config.py — Settings model
- 5.6 health.py
- 5.7 logging.py
- 5.8 ports.py — PortAllocator
- 5.9 recovery.py
- 5.10 runtime.py — RuntimeState
- 5.11 service.py — Service
- 5.12 systemd.py

6. Package sqtseries.engine

- 6.1 engine/__init__.py
- 6.2 engine/db.py — Database, Connection, Result
- 6.3 engine/store.py — StorageEngine
- 6.4 engine/schema.py
- 6.5 engine/pragmas.py
- 6.6 engine/migrations.py
- 6.7 engine/checkpoint.py — CheckpointManager
- 6.8 engine/backup.py — BackupManager
- 6.9 engine/maintenance.py — MaintenanceManager

7. Package sqtseries.partition

- 7.1 partition/__init__.py
- 7.2 partition/manager.py — PartitionManager
- 7.3 partition/retention.py — RetentionPolicy/Manager
- 7.4 partition/rollup.py — rollup functions + RollupManager

8. Package sqtseries.query

- 8.1 query/__init__.py
- 8.2 query/agg.py — aggregations, downsampling, gap fill
- 8.3 query/builder.py — TimeSeriesDB

9. Package sqtseries.messaging

- 9.1 messaging/__init__.py

9.2 messaging/protocol.py

9.3 messaging/context.py

9.4 messaging/ingress.py — Ingress

9.5 messaging/broker.py — QueryBroker

9.6 messaging/admin.py — AdminBroker

9.7 messaging/pubsub.py — PubSub, SubscriptionTracker

9.8 messaging/connection_registry.py — ConnectionRegistry

9.9 messaging/stats_publisher.py — StatsPublisher

9.10 messaging/worker.py — WorkerPool

10. Package sqtseries.gateway

- 10.1 gateway/__init__.py
- 10.2 gateway/app.py — create_app
- 10.3 gateway/routes.py — REST endpoints
- 10.4 gateway/ratelimit.py — RateLimitMiddleware
- 10.5 gateway/websocket.py — subscribe_and_forward

11. Examples

- 11.1 producer.py
- 11.2 consumer.py
- 11.3 stats_monitor.py
- 11.4 ingest_http.py
- 11.5 admin_examples.py
- 11.6 stream_examples.py
- 11.7 tags_examples.py
- 11.8 run_custom.py
- 11.9 maintenance_examples.py
- 11.10 query_examples.py
- 11.11 question_examples.py
- 11.12 question_examples_advanced.py

12. Camera Analytics Stack (examples/camera)

- 12.1 overlay_server.py
- 12.2 camera_feed.py
- 12.3 overlay.html
- 12.4 conn_validate.py
- 12.5 sqtcam.sh

13. scripts/benchmark.py

14. Tests (54 files + confest.py)

- 14.1 confest.py
- 14.2 Test files by area

15. docs/ Directory

16. Wire Protocol Reference

17. End-to-End Data Flows

18. Glossary

1. Project Overview

sqtseries is a local, single-file time-series database. You feed it measurements — `metric` + `value` + optional string `tags` — and later ask questions like "average CPU over the last hour" or "99th percentile response time this week". Everything lives in one SQLite file (`~/.sqtseries/data/db.sqlite` by default), so you can copy, back up, or inspect it like any file.

Two transports expose the same JSON wire format:

- **ZeroMQ** — PUSH ingest (12501), REP queries (12502), XPUB streaming (12503), REP admin (12504), PUB stats events (12506). High throughput, any language with a ZMQ binding.
- **HTTP** — FastAPI REST + WebSocket on 12505. Works from curl, browsers, and any language.

Performance comes from two storage tricks: monthly *partitions* (`measurements_YYYY_MM` WITHOUT ROWID tables) and *hourly rollups* (a background task pre-aggregates completed hours into `rollup_hourly` so wide queries read a few rows instead of millions). A query fast path merges rollup rows with the raw edges of the window — exact same answer, about 9× faster.

Project layout

Location	Contents	Lines
<code>src/sqtseries/</code>	43 Python source files across 6 packages	~6,100
<code>examples/</code>	12 standalone runnable scripts + camera stack	~4,200
<code>scripts/</code>	1 reproducible benchmark	182
<code>tests/</code>	54 test modules + <code>confest.py</code> (unit, integration, property, fuzz, stress, Playwright)	~8,500
<code>docs/</code>	Hand-written HTML documentation site (not generated)	—

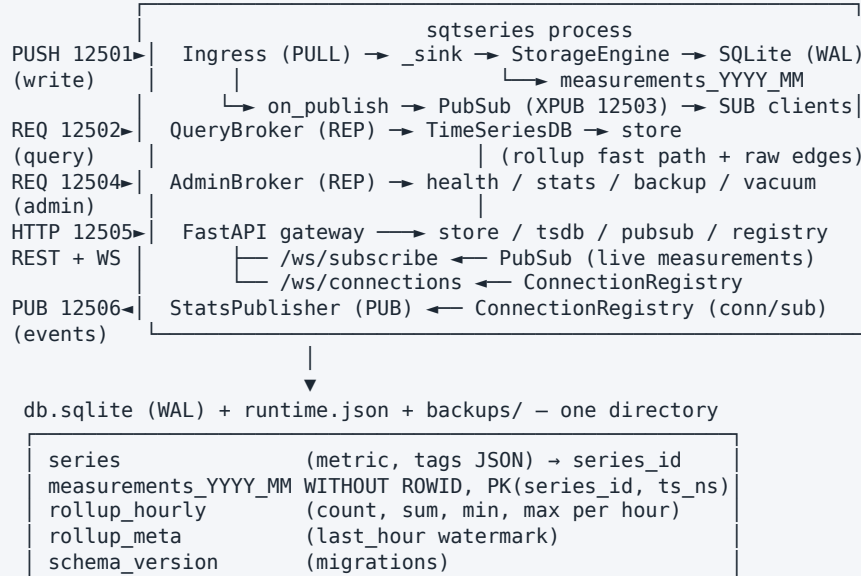
Runtime dependencies

Package	Role
<code>orjson</code>	Fast JSON (de)serialization for the wire protocols and tag encoding
<code>pyzmq</code>	ZeroMQ message bus: PUSH / REP / XPUB / PUB sockets
<code>fastapi</code> + <code>uvicorn</code>	HTTP REST + WebSocket gateway
<code>httpx</code>	HTTP client (tests, <code>conn_validate.py</code>)
<code>click</code>	CLI framework
<code>pydantic</code> / <code>pydantic-settings</code>	Configuration model + env/file loading
<code>structlog</code>	Structured logging (JSON or console)

Dev extras: `pytest` + `pytest-asyncio/timeout/cov`, `hypothesis` (property tests), `ruff/black/isort`, `playwright` (dashboard page tests), `memory-profiler` (bench).

2. Architecture at a Glance

Everything runs in *one process*: the event loop drives the ZMQ sockets and the HTTP gateway reads the same in-memory engine objects (`StorageEngine` , `TimeSeriesDB` , `PubSub`). There is no service-to-service hop.



Package responsibilities

Package	Responsibility
<code>sqtseries</code> (top)	Public API (<code>Client</code>), CLI, config, service orchestration, health/logging/ports/runtime/systemd
<code>sqtseries.engine</code>	Raw sqlite3 layer: schema, partition DDL, PRAGMAs, migrations, checkpoints, backups, maintenance
<code>sqtseries.partition</code>	Partition lifecycle: monthly tables, retention drops, hourly rollup aggregation
<code>sqtseries.query</code>	Query facade: aggregation functions, downsampling, gap filling, rollup fast path
<code>sqtseries.messaging</code>	ZeroMQ layer: protocol validation, ingest, query/admin brokers, pub/sub, connection registry, stats, worker pool
<code>sqtseries.gateway</code>	FastAPI REST + WebSocket gateway: routes, rate limiting, WS streaming, security headers

Background tasks (started by `Service._start`)

Manager	Schedule	What it does
<code>CheckpointManager</code>	every 60 s	TRUNCATE-checkpoints the WAL when it exceeds 64 MB; tracks readers blocking checkpoints
<code>RollupManager</code>	every 5 min (<code>rollup.interval</code>)	Aggregates freshly-completed hours into <code>rollup_hourly</code> ; honors the clock-skew guard
<code>MaintenanceManager</code>	hourly	Re-runs <code>ANALYZE</code> to refresh query-planner statistics
<code>RetentionManager</code>	hourly (<code>retention.check_interval</code>)	Drops monthly partitions older than the TTL (default 30 d), and their rollup rows
<code>BackupManager</code>	every 24 h, first pass on boot	Writes a consistent <code>VACUUM INTO</code> snapshot to <code>backup.path</code>
<code>StatsPublisher</code>	every 10 s + on events	Publishes <code>conn</code> / <code>sub</code> / <code>report</code> events on port 12506

Fixed ports (all bound to 127.0.0.1)

Port	ZMQ socket	Purpose	Config key
12501	PULL	Ingest measurements	<code>ingestion.port</code> (auto-detected from 12500–12700 by default!)
12502	REP	Run queries	<code>query.port</code>
12503	XPUB	Live streaming + subscriber tracking	<code>streaming.port</code>
12504	REP	Admin commands	<code>admin.port</code>
12505	—	HTTP REST + WebSocket	<code>http.port</code>
12506	PUB	Connection/subscription events	<code>stats.port</code>

Ingest auto-detection: with `ports.auto_detect = true` (the default) the ingest port is the first free port in `[12500+offset, 12700+offset]` that is not one of the fixed ports — so it is usually **12500**, not the configured 12501. The active ports are written to `runtime.json` next to the database; `sqtseries ports` / `sqtseries status` read them back.

3. Core Concepts

3.1 Measurement → series → partition

- A **measurement** is one number: `metric` (string), `value` (finite number), optional `tags` (string → string map), optional `timestamp` (epoch *seconds*; server time when omitted).
- Internally storage uses nanosecond epoch integers (`timestamp_ns`); the wire protocol sends seconds, ZMQ queries take nanosecond bounds, and HTTP uses seconds.
- A **series** is the unique pair (`metric`, `tags`). It gets an integer `series_id` from the `series` table (unique index on (`metric`, `tags`)). Writing the same (`metric`, `tags`) again appends to the same series; different tag sets are different series.
- Measurements are stored in **monthly partitions** named `measurements_YYYY_MM`, created lazily on first write. Partitions are `WITHOUT ROWID` tables with the clustered primary key (`series_id`, `timestamp_ns`) plus a secondary index on `timestamp_ns` (cross-series time ranges and retention).

3.2 Tags: dimensions, not IDs

Tags are dimensions you slice by: `{"host": "web1", "env": "prod"}`. Every distinct tag set creates a new series, so **never tag high-cardinality values** (request IDs, session IDs, timestamps) — they multiply series and slow per-series queries. Tag values must be strings; non-string values are rejected (HTTP 400 / `ProtocolError`). Wire queries over a metric merge *all* of its tag sets; per-series breakdowns need the embedded Python API (`StorageEngine.series_ids_for_metric` + `get_series_meta`).

3.3 Hourly rollup

- A background task rolls *completed* hours into `rollup_hourly` (`series_id`, `hour_start_ns`, `count`, `sum`, `min`, `max`; `avg = sum/count`).
- Only hours that ended at least `reject_client_timestamp_skew_s` ago are rolled, so a "late" write (a client clock within the skew window) can never land in a rolled hour — kept in-order by the clock-skew guard on ingest.
- A `last_hour` watermark in `rollup_meta` records how far the rollup advanced; each run scans only the data since the previous run, and `INSERT OR REPLACE` keeps re-runs idempotent.
- The current (in-progress) hour is always read live from the raw partitions, so results are never stale.

3.4 Query fast path

`TimeSeriesDB.query` decides whether a query can be answered from the rollup (`_rollup_eligible`): the aggregation must be expressible from count/sum/min/max (`avg`, `sum`, `min`, `max`, `count`), the interval must be a whole-hour multiple (or None), the window must contain at least one fully-inside rolled hour, and the tail must not extend past the watermark. The eligible window is split into three disjoint sources — raw left edge hour, rollup middle hours, raw right tail — merged per bucket, so nothing is counted twice. `median`, `p95`, `p99`, `first`, `last` always take the raw path.

3.5 Retention

Retention drops *whole monthly partitions* whose *last day* falls before `now - ttl` (fast `DROP TABLE`), and deletes those hours from `rollup_hourly` so deleted data leaves no aggregate trace. TTL syntax: `30d`, `12h`, `4w`, `90s`, `45m`.

3.6 Crash safety

- SQLite WAL journal: every write is a single atomic transaction (`BEGIN IMMEDIATE`).
- Startup: `PRAGMA quick_check`, then `recover_wal` (re-assert WAL mode + a PASSIVE checkpoint). A failed integrity check logs the problems; `strict` mode raises.
- Shutdown: `PRAGMA optimize`, then a TRUNCATE checkpoint so the next start is fast.
- Backups use `VACUUM INTO` — a consistent snapshot even while writes are in flight.

3.7 Configuration precedence

Defaults < config file (TOML/YAML/JSON, `SQT_SERIES_CONFIG_FILE` or `--config`) < environment variables (`SQT_SERIES_<SECTION>__<KEY>`, e.g. `SQT_SERIES_DATABASE__PATH`). Env always wins over file values — the loader computes the set of env-set leaf paths and only merges file keys the env did *not* set.

4. Root Files

pyproject.toml packaging + tool config

- **Build:** setuptools ≥ 69 ; package discovery under `src/`.
- **Metadata:** name `sqtseries`, version `0.1.0`, MIT license, requires-python ≥ 3.14 , alpha status.
- **[project.scripts]:** `sqtseries = "sqtseries.cli:main"` — the console entry point.
- **[tool.ruff] / [tool.black]:** line-length 88, target py314. Lint selection includes E/F/W/I (pycodestyle/flake8), UP (pyupgrade), B (bugbear), C4, SIM, RET, PTH, S (bandit), RUF, PERF. Per-file ignores: `tests/*` (S101/S105/S106/S110/S603), `examples/*` (S310/S311/S603/PERF401/RUF005/PTH202 — they launch the CLI by design), `overlay_server.py` (S104 — LAN accessibility by design), `systemd.py` (S603/S607 — systemctl via PATH), `benchmark.py` / `stress.py` (S108). `test_backup/test_concurrency` ignore S608 (server-derived table names).
- **[tool.pytest.ini_options]:** testpaths `tests`, `asyncio_mode = "auto"` (async tests run without markers), 60 s global timeout.
- **[project.optional-dependencies]:** `dev` (pytest stack, ruff/black/isort, httpx2, playwright) and `bench` (memory-profiler).

requirements.txt pinned core deps

Pinned versions of the runtime dependencies (orjson 3.11.9, pyzmq 27.1.0, fastapi 0.141.1, uvicorn 0.52.1, httpx 0.28.1, click 8.4.2, pydantic 2.13.4, pydantic-settings 2.14.2, structlog 26.1.0) for reproducible installs without the optional dev/bench groups.

README.md project front page

Positioning, use cases, quick start (3 lines of Python or 2 curl commands), design highlights (single-file storage, hourly summaries, two transports, live connection tracking, crash safety, hardened systemd, ingest port auto-detection), the port table, CLI command list, admin command list, and links to the docs site. Two behavioral claims worth knowing: `run` refuses to start over a database another live instance serves, and `stop` refuses to touch an instance whose database differs from the resolved one.

.gitignore ignore rules

Excludes Python artifacts (`__pycache__`, `.pyc`, `venvs`), build output, `.pytest_cache`, coverage data, and local work directories such as `sqt-custom-work/` and `/tmp`-style scratch (as produced by `run_custom.py`).

5. Package sqtseries (top level)

src/sqtseries/__init__.py^{5 lines}

Public entry point. Re-exports the client types and the version:

```
__version__ = "0.1.0"
```

```
__all__ = ["Client", "ClientError", "__version__"]
```

```
from sqtseries import Client
```

is the documented usage.

src/sqtseries/__main__.py^{4 lines}

Enables `python3 -m sqtseries ...` — identical to the `sqtseries` console script. Just imports `main` from `cli` and runs it under `__main__`.

src/sqtseries/client.py^{262 lines}

High-level Python client for a *running* service. Wraps the ZeroMQ transports: PUSH (write, 12501), REQ (query, 12502), SUB (subscribe, 12503), REQ (admin, 12504). Uses the same JSON wire format as the HTTP gateway.

Module constants

```
DEFAULT_HOST = "127.0.0.1"
```

```
DEFAULT_PORTS = {"write": 12501, "query": 12502, "subscribe": 12503, "admin": 12504}
```

```
RCV_TIMEOUT_MS = 30_000
```

— query/admin replies time out to `ClientError` instead of hanging.

```
WRITE_LINGER_MS = 2000
```

— the write PUSH socket flushes queued measurements for 2 s on close; zero would drop a measurement written immediately before exit.

Class `Client`

Usage: `Client()`, `client.write("cpu.usage", 0.72, {"host": "web1"})`, `client.query("cpu.usage", start=..., end=...)` (nanoseconds), iterate `client.subscribe("cpu.")`. Supports the context manager protocol.

METHOD

```
__init__(self, host="127.0.0.1", ports: dict | None = None)
```

Stores host/port map (merging over `DEFAULT_PORTS`) and lazily opens sockets via the `_get_*_sock` helpers. All four sockets are created on demand from one `zmq.Context()`.

METHOD

```
write(metric, value, tags=None, timestamp=None) -> None
```

Send one measurement (PUSH, fire-and-forget). Optional `timestamp` is epoch seconds. Serialized with orjson.

METHOD

```
write_many(points: list[dict]) -> None
```

Send multiple raw point dicts, one frame each (a convenience loop, not a server-side batch).

METHOD

```
query(metric, start=None, end=None, *, aggregation=None, interval=None, limit=None, order="asc") -> list[dict]
```

Run a query and return `[{timestamp, value}, ...]` with timestamps in *seconds* (the server converts ns → s). `start / end` are epoch *nanoseconds*. On a recv timeout it replaces the broken REQ socket (REQ enforces strict send/recv alternation — a timed-out recv leaves the socket unusable, so `_reset_sock` closes it and the next call recreates it) and raises `ClientError("query timed out...")`.

METHOD

```
aggregate(metric, start=None, end=None, *, funcs=None) -> dict[str, float]
```

Compute one or more aggregations over a window; returns `{func: value}`. Sends `{"type": "query", "metric": ..., "aggregations": "avg,min,..."}.`

METHOD

```
admin(cmd, **kwargs) -> dict
```

Send an admin command to port 12504: `ping / health / stats / optimize / backup / vacuum / connections / conncheck / subscribers`. Extra kwargs become part of the request, e.g. `admin("conncheck", ids=["abc"])`. Error replies raise `ClientError`.

METHOD

```
subscribe(topic="*", timeout=0.5) -> Iterator[dict | None]
```

Yield live measurement dicts matching `topic` (SUB socket). Polls with a timeout and yields `None` as a heartbeat when idle, so callers can break out of the loop cleanly. Frames are `[topic, payload]`; yields the parsed payload. The poll-then-`recv_multipart(NOBLOCK)` pattern is per pyzmq docs (avoids blocking recv vs context close).

METHOD

```
_get_write_sock/_get_query_sock/_get_sub_sock/_get_admin_sock() -> zmq.Socket
```

Lazy socket factories. Write: PUSH, SNDHWM 10000, LINGER 2000. Query/Admin: REQ, LINGER 500, RCVTIMEO 30 s. Subscribe: SUB, LINGER 0.

METHOD

```
_reset_sock(attr)
```

Close a broken socket so the next call recreates it. Only REQ sockets strictly need this (strict alternation after a recv timeout); harmless for the others.

METHOD

```
close()
```

Close all sockets and terminate the context (idempotent). The write socket keeps its flush linger so queued-but-undelivered measurements are still sent.

METHOD

```
__enter__ / __exit__
```

Context-manager support; exit calls `close()`.

Class `ClientError(Exception)`

Raised when the service returns an error reply, or a query/admin call times out.

`src/sqtseries/cli.py` 344 lines

The command-line interface (click). The `sqtseries` console script points here. Global flags `--config PATH` and `--db PATH` must come *before* the subcommand.

Helper functions

FUNCTION

```
_load(config_file, db_path=None) -> Settings
```

Load `Settings`, apply the `--db` override, run `validate_settings`, and raise `click.ClickException` listing the errors on failure.

FUNCTION

```
_runtime_path(settings) -> str
```

`runtime.json` lives next to the database file.

FUNCTION

```
_load_runtime(config_file, db_path=None) -> dict | None
```

Read the runtime file if present (None when absent/corrupt).

FUNCTION

```
_pid_is_sqtseries(pid) -> bool
```

Whether `/proc/<pid>/cmdline` mentions "sqtseries" — used to detect stale runtime files from other programs.

Command group `main`

Click group with `--version`, global `--config / --db` options stored on the context object.

Commands

COMMAND

```
run
```

Start the service in the foreground. Guards: refuses to start when the runtime file says another live sqtseries process already serves the same database (or a different one is being served — a conflict). Prints a note if other `*.sqlite` files sit in the same directory. Configures logging, builds `Service(settings)`, and runs `start → run → shutdown` under `asyncio.run`, suppressing KeyboardInterrupt (clean SIGINT shutdown path).

COMMAND

```
stop
```

Send SIGTERM to the pid in `runtime.json` (validated: must be a live sqtseries process, and the runtime's `db_path` must match what this invocation resolves — refuses to stop the wrong instance). Then waits up to 10 s for the runtime file to disappear (clean shutdown removes it last) or the pid to vanish, so a restart script cannot race a still-dying process.

COMMAND

```
status
```

Show running/not-running, version, database path, and every active port from `runtime.json`; warns when the config resolves to a different database than the running instance.

COMMAND

```
ports
```

Show active ports from `runtime.json`, or the configured ports when not running.

COMMAND

```
health
```

Open the database read-only and run `quick_check`; exit code 0 = ok, 1 = degraded.

COMMAND`stats`

Open the database and print metric and series counts (`StorageEngine.list_metrics` , `series_count`).

COMMAND`vacuum`

Full `VACUUM` — requires the service to be stopped (VACUUM can't run inside a transaction / under active connections).

COMMAND`optimize`

`PRAGMA optimize` — refresh query-planner statistics.

COMMAND`backup`

`backup_database(engine, settings.backup.path)` — consistent VACUUM INTO snapshot; prints the written path.

COMMAND`install [--system]`

Install the systemd unit (user-level by default; `--system` needs root). Embeds the current interpreter and `--config` if one was passed, plus db path and backup path.

COMMAND`uninstall [--system]`

Disable and remove the systemd unit.

`src/sqtseries/config.py` 410 lines

Configuration model. Supports TOML / YAML / JSON files and environment variables (`SQT_SERIES_*`); env wins over file values. All settings are pydantic models.

Module constants

`PORT_RANGE_START = 12500` / `PORT_RANGE_END = 12700`

Settings models (each is a `BaseModel`)**CLASS**`DatabaseSettings`

SQLite storage: `path` (`~/.sqtseries/data/db.sqlite`), `page_size` 8192, `cache_size` -64000, `mmap_size` 256 MiB, `busy_timeout` 5000, `journal_size_limit` 64 MiB, `threads` 4, `batch_size` 2000, `flush_interval` 1.0.

CLASS`IngestionSettings`

`port` 12501, `hwm` 10000, `pending_max` 100, `max_message_size` 50 MiB, `reject_client_timestamp_skew_s` 300.0.

CLASS`QuerySettings`

`port` 12502, `timeout_s` 30.0 (also used for the HTTP gateway cap).

CLASS`StreamingSettings`

`port` 12503, `linger_seconds` 30.0 (topic linger after the last unsubscribe).

CLASS`StatsSettings``enabled` `True`, `port` `12506`.**CLASS**`AdminSettings``port` `12504`.**CLASS**`HttpSettings``port` `12505`, `host` `127.0.0.1`, `cors_origins` `["*"]`, `rate_limit_per_minute` `600`, `max_websocket_connections` `1000` (excess closes with 1013).**CLASS**`LoggingSettings``level` `INFO`, `format` `"console" or "json"`, `file` optional path (10 MB rotating, 5 backups).**CLASS**`RetentionSettings``enabled` `True`, `default_ttl` `"30d"`, `check_interval` `"1h"`, `partition_interval` `"month"` (only "month" supported).**CLASS**`RollupSettings``enabled` `True`, `interval` `"5m"` (how often completed hours are rolled).**CLASS**`MaintenanceSettings``enabled` `True`, `analyze_interval` `"1h"`.**CLASS**`BackupSettings``enabled` `True`, `interval` `"24h"`, `path` `"~/sqtseries/backups/"`.**CLASS**`PortsSettings``port_range_start` / `port_range_end` `12500–12700`, `auto_detect` `True`, `port_offset` `0`.**CLASS**`Settings(BaseSettings)`

Top-level settings. Env prefix `SQT_SERIES_`, nested delimiter `_`, `env_ignore_empty=True` (VAR= means "unset"), case-insensitive. Holds every section above plus `config_file`. Key methods:

- `load(config_file=None) -> Settings` — precedence: defaults < file < env < init kwargs. Implementation detail: pydantic-settings gives init kwargs priority over env, so the loader instantiates twice — first with env applied, then re-instantiates with only the file leaf paths that the environment did *not* set.
- `db_path_expanded() -> str` — database path with `~` expanded.

Module functions

FUNCTION

```
_enviro_leaf_paths() -> set[tuple[str, ...]]
```

Scans `os.environ` for `SQT_SERIES_*` keys, converts `SQT_SERIES_DATABASE__BATCH_SIZE` → `("database", "batch_size")`, normalizes case, excludes the root `CONFIG_FILE` key.

FUNCTION

```
_iter_leaf_paths(mapping, prefix=())
```

Yield `(path, value)` for every leaf of a nested mapping (recursive).

FUNCTION

```
_set_path(target, path, value)
```

Set `target[path] = value`, creating intermediate dicts.

FUNCTION

```
_read_config_file(path) -> dict
```

TOML (`tomllib`), YAML (requires `pyyaml`, else a clear `ValueError`), or JSON (`orjson`); any other suffix raises. JSON/YAML must be a top-level mapping.

FUNCTION

```
validate_settings(settings) -> list[str]
```

Returns a list of configuration errors (empty when valid): ports in 1–65535 and unique, `hwm/pending_max/batch_size` ≥ 1, `batch_size` ≤ 8191 (SQLite_MAX_VARIABLE_NUMBER safety), `max_websocket_connections` ≥ 1, and duration strings parse correctly (`rollup.interval`, `maintenance.analyze_interval`, `retention` TTLs, `backup.interval`) — each validated only when its feature is enabled.

src/sqtseries/health.py 31 lines

FUNCTION

```
collect_health(db, *, started_at, version="0.1.0") -> dict
```

Assemble the health payload: `status` ("ok" or "degraded"), `version`, `uptime` (seconds since `started_at`), `database` (bool `quick_check == "ok"`; any exception → False). Used by the admin `health` command and `Service.health()`.

FUNCTION

```
is_ready(db) -> bool
```

Boolean quick check for the HTTP `/api/v1/health` endpoint.

src/sqtseries/logging.py 78 lines

Structured logging via structlog over stdlib logging: JSON output for production, pretty console for development, optional rotating file output.

FUNCTION

```
_json_serializer(obj, **_) -> str
```

`orjson`-backed serializer for the JSON renderer.

FUNCTION

```
configure_logging(settings: LoggingSettings) -> BoundLogger
```

Builds the processor chain: contextvars merge → add_log_level → ISO UTC timestamps → StackInfoRenderer → format_exc_info (plain strings so both renderers can display tracebacks) → JSONRenderer or ConsoleRenderer (colors when stderr is a tty). Configures the stdlib root logger, adds a 10 MB / 5-backup RotatingFileHandler when `settings.file` is set (skips if a handler for that file already exists — avoids duplicates on repeated calls), then calls `structlog.configure` and returns `structlog.get_logger("sqtseries")`.

src/sqtseries/ports.py 71 lines

Port allocation within a range (12500–12700 default), used for ingest auto-detection. Finds a free port by binding a socket and closing it — race-tolerant.

EXCEPTION

```
PortInUseError
```

Requested fixed port unavailable.

EXCEPTION

```
PortExhaustedError
```

No free port in the range.

CLASS

```
PortAllocator (dataclass)
```

Fields: `start` 12500, `end` 12700, `offset` 0, `reserved` set (ports bound to other fixed services), private `_used` set.

- `from_settings(settings: PortsSettings) -> PortAllocator` — classmethod build.
- `alloc(*, auto_detect=True) -> int` — with `auto_detect` False returns `start+offset` if available; otherwise scans upward for the first port not in `_used` / `reserved` that binds successfully. Raises `PortInUseError` / `PortExhaustedError`.
- `release(port)` — forget a port (back to the pool).
- `_is_available(port) -> bool` — static: tries `bind(("127.0.0.1", port))`.

src/sqtseries/recovery.py 41 lines

WAL recovery and corruption detection at startup.

EXCEPTION

```
CorruptionError
```

Raised when the database fails integrity checks in strict mode.

FUNCTION

```
check_integrity_on_startup(db, *, strict=False) -> int
```

Runs `PRAGMA quick_check`. Clean → 0. Otherwise logs the problem count, runs the full `PRAGMA integrity_check` logging every line, and raises `CorruptionError` when `strict=True`; returns the number of problems otherwise.

FUNCTION

```
recover_wal(db)
```

Ensures WAL journal mode is active (switches if the file was last used in another mode) and runs a PASSIVE checkpoint to compact the log at startup.

src/sqtseries/runtime.py 56 lines

Runtime state file: PID, ports, start time, database path. Written after startup, removed on shutdown. Clients (`sqtseries` `status`, `ports`, `stop`, `run` conflict detection) use it to discover the active ports.

CLASS`RuntimeState`

`__init__(path)` stores the path.

- `write(*, pid, ports, db_path, version="0.1.0")` — writes `{pid, ports, db_path, version, started_at}` atomically (tmp file + `replace`).
- `read() -> dict | None` — orjson parse; malformed/missing → `None`.
- `remove()` — delete the file, suppressing `OSError`.
- `exists` — property; whether the file exists.

src/sqtseries/service.py 566 lines — the orchestrator

Service manager: startup sequence, main event loop, graceful shutdown. Wires every component together: engine → managers → sockets → HTTP gateway.

EXCEPTION`ServiceError`

Raised for invalid configuration at construction.

CLASS`Service`

Holds `settings`, `started_at`, and one attribute per component: `engine`, `store`, `ts`, `ingress`, `broker`, `admin_broker`, `pool`, `checkpoint_manager`, `rollup_manager`, `maintenance_manager`, `retention_manager`, `backup_manager`, `pubsub`, `stats_publisher`, `http_server/http_task`, `_publish_tasks` set, `runtime` (`RuntimeState`), `_runtime_owned` flag, `connection_registry`.

METHOD`async start()`

Calls `_start()`; on any failure runs `shutdown()` to clean up whatever started, then re-raises.

METHOD

```
async _start()
```

The boot sequence, in order:

1. Open the engine (`create_sqlite_engine`), run migrations, check integrity (non-strict), recover WAL.
2. Build `StorageEngine` + `TimeSeriesDB`.
3. Start each enabled background manager (checkpoint, rollup with the skew window, maintenance, retention, backup).
4. Start `PubSub` (XPUB) wired to the connection registry.
5. Allocate the ingest port via `PortAllocator` — with `auto_detect`, it must avoid the five fixed ports (stats included), then start `Ingress` with `_sink` and `_on_publish` callbacks.
6. Start the query broker (REP) and admin broker (REP, no handler timeout — admin ops such as backup/vacuum may run long).
7. Start the `WorkerPool` (size 1) whose step pumps all three ZMQ sockets with `block=False` and reports whether any had work (the worker then sleeps 0 vs 10 ms — avoids GIL starvation of CPU-bound threads).
8. Start the stats publisher (if enabled), then the HTTP gateway.
9. Run `ANALYZE` once (only after every socket bound, so failed boots in a restart loop don't hammer planner stats).
10. Write runtime.json with the real ports; set `_runtime_owned = True`.

METHOD

```
async _start_http_gateway()
```

Builds the FastAPI app via `create_app`, runs uvicorn's `Server.serve()` as an async task sharing the loop, and waits up to 5 s for `started`. uvicorn is configured with `log_config=None` / `access_log=False` (structlog owns logging), `timeout_graceful_shutdown=5`, `ws_max_size=16 MiB`, `ws_ping_interval=20`, `ws_ping_timeout=20`. Note: uvicorn 0.52 installs its own signal handlers, but `Service.run()`'s `add_signal_handler` replaces them, so `should_exit` is only ever set in `shutdown()`. Bind failure surfaces a clean `RuntimeError`.

METHOD

```
_sink(metric, tags, value, ts_ns)
```

Persist one ingested measurement immediately via `store.insert_many`. There is no app-level write queue: SQLite serializes writers itself (single-writer + BEGIN IMMEDIATE + busy_timeout), so a queue would only add latency.

METHOD

```
_on_publish(topic, payload)
```

Fire an async task per measurement to republish to live subscribers; tasks are tracked in `_publish_tasks` and drained on shutdown.

METHOD

```
_query_handler(query) -> dict
```

Serves the query REP socket: validates `limit`, dispatches the `aggregations` (comma-separated) form to `ts.aggregate`, otherwise to `ts.query`, converts ns → seconds timestamps, and maps value/type errors to `INVALID_QUERY`.

METHOD

```
_admin_handler(query) -> dict
```

Serves admin commands: `ping`, `health`, `stats`, `optimize`, `backup`, `vacuum` (VACUUM_BUSY error shape when it fails while running — recommends `sqtseries vacuum` stopped), `connections`, `conncheck`, `subscribers`. Unknown commands raise `ProtocolError` → `INVALID_REQUEST` reply.

METHOD

```
_admin_stats() -> dict
```

Assembles live counters from every component: ingested/invalid/ingest_errors, queries, admin_requests, wal_bytes/checkpoint_busy_runs/checkpoints, analyze_runs, retention_runs/partitions_dropped, backup_runs/backups_created/last_backup, published/subscribers, series/metrics, uptime_s.

METHOD

```
_admin_connections / _admin_conncheck(ids) / _admin_subscribers
```

Delegates to the ConnectionRegistry: active WebSocket list; which of the given IDs are present; per-topic ZMQ subscriber counts.

METHOD

```
health() -> dict
```

Wraps `collect_health(self.engine, started_at=...)`.

METHOD

```
async run()
```

Waits for SIGINT/SIGTERM using an `asyncio.Event` (not `loop.stop()` — stopping the loop mid-run makes `asyncio.run` raise, which systemd's `Restart=on-failure` would treat as a crash). Falls back to `signal.signal` when `add_signal_handler` is unavailable (non-main thread).

METHOD

```
async shutdown()
```

Graceful shutdown, in dependency order: stop background managers, the worker pool, the three ZMQ sockets, ask uvicorn to exit (`should_exit = True`), drain in-flight publishes *before* stopping pubsub, stop pubsub and stats publisher, close the store, run `optimize` + TRUNCATE checkpoint on the engine, dispose it, and remove `runtime.json` *only if this instance wrote it* (a failed second start must not delete a healthy instance's state).

src/sqtseries/systemd.py^{101 lines}

systemd unit management (user or system service).

```
UNIT_NAME = "sqtseries.service"
```

FUNCTION

```
unit_template_contents(python, db_path, backup_path=None, config_file=None) -> str
```

Renders the unit file. `ExecStart` is `<python> -m sqtseries [--config <path>] run` (shlex-joined). Hardening: `NoNewPrivileges=true`, `ProtectSystem=strict`, `ProtectHome=read-only`, and `ReadWritePaths` limited to the db directory (+ backup path). Environment sets `SQT_SERIES_DATABASE_PATH` and `SQT_SERIES_LOGGING_LEVEL=INFO`. `Restart=on-failure`, `RestartSec=5`, `StartLimitBurst=3`.

FUNCTION

```
_unit_path(system) -> Path
```

`/etc/systemd/system` vs `~/.config/systemd/user`.

FUNCTION

```
_systemctl(args)
```

Best-effort `systemctl` invocation (captures output, ignores failure — testable).

FUNCTION

```
install_systemd_unit(python=None, system=False, db_path=None, backup_path=None, config_file=None)
-> Path
```

Writes the unit file and runs `daemon-reload` + `enable --now` (`--user` prefix when not system). Returns the unit path.

FUNCTION

```
uninstall_systemd_unit(*, system=False) -> Path
```

`disable --now` then unlink the unit file.

6. Package sqtseries.engine

The raw sqlite3 storage layer: schema, partitions, PRAGMAs, migrations, checkpoints, backups, maintenance. Why raw sqlite3 (per the module docs): benchmark-validated 2026-08-07 — roughly 6.7× faster point lookups and 1.2× bulk inserts than alternatives on the exact hot paths; aiosqlite rejected (sequential thread hops).

src/sqtseries/engine/__init__.py^{40 lines}

Re-exports the public engine API: `BackupExistsError`, `BackupManager`, `CheckpointManager`, `Connection`, `Database`, `MaintenanceManager`, `SeriesNotFoundError`, `StorageEngine`, `backup_database`, `backup_latest`, `create_sqlite_engine`, `get_schema_version`, `incremental_vacuum`, `initialize_schema`, `integrity_check`, `quick_check`, `run_analyze_once`, `run_migrations`, `run_optimize`, `upgrade`, `wal_checkpoint`.

src/sqtseries/engine/db.py^{248 lines}

The raw sqlite3 layer. Reader/writer split: `connect()` gives a read-only connection that never checkpoints the WAL; `begin()` gives a writer transaction using `BEGIN IMMEDIATE` (takes the write lock up front — under WAL a deferred read → write upgrade returns `SQLITE_BUSY` immediately, which `busy_timeout` cannot retry).

Module constants

`CONNECTION_PRAGMAS` — applied to every new connection: `journal_mode WAL`, `synchronous NORMAL`, `cache_size -64000`, `mmap_size 256 MiB`, `busy_timeout 5000`, `temp_store MEMORY`, `foreign_keys ON`, `locking_mode NORMAL`, `journal_size_limit 64 MiB`, `threads 4`, `analysis_limit 400`.

`READER_AUTOCHECKPOINT = 0` / `WRITER_AUTOCHECKPOINT = 10000` — readers never trigger checkpoints (prevents reader-writer contention); the writer checkpoints every ~80 MB at 8192-byte pages (10× fewer fsync spikes than the default 1000 pages).

Class `Result`

Thin wrapper over `sqlite3.Cursor`. `__slots__` for memory. Methods: `fetchall()`, `fetchone()`, `first()`, `scalar()` (first column of first row), properties `lastrowid` / `rowcount`, `__getitem__(idx)` (`fetchone` + `IndexError`), `__iter__` (iterates rows).

Class `Connection`

Wraps a raw connection; context manager commits on clean exit and always closes. `execute(sql, params)` / `exec_driver_sql` / `executescript` / `executemany` all return a `Result`; `dbapi_connection` property exposes the raw object; `commit()` / `rollback()` / `close()` delegate. Cursors are closed on `sqlite3.Error` before re-raising.

Class `Database`

CONTEXTMANAGER

```
connect(apply_pragmas=True) -> Connection
```

Fresh `READER` connection (`autocheckpoint 0`, `transaction=False`).

CONTEXTMANAGER

```
begin() -> Connection
```

`WRITER` transaction: `BEGIN IMMEDIATE`, commits on clean exit, rolls back on any exception, closes in finally.

METHOD

```
dispose()
```

No persistent handles (connections are per-use), so a no-op kept for API compatibility.

METHOD

```
get_table_names() -> list[str]
```

Table names from `sqlite_master`, sorted.

METHOD

```
execute(sql, params=None) -> Result
```

Executes inside a committed writer transaction — writes and DDL persist (the old reader-based implementation silently rolled back writes on close).

METHOD

```
scalar(sql, params=None)
```

Run and return the first column of the first row.

METHOD

```
_open(apply_pragmas=True, autocheckpoint=0)
```

Opens `sqlite3.connect(check_same_thread=False)`, applies `foreign_keys` + each `CONNECTION_PRAGMA` + `wal_autocheckpoint`.

FUNCTION

```
create_sqlite_engine(path, settings=None) -> Database
```

Create a Database. On a *fresh file*, applies the one-time file-level PRAGMAs (`page_size = 8192`, `auto_vacuum = INCREMENTAL`) *before* WAL — per sqlite.org, `auto_vacuum/page_size` can only change outside WAL mode, so the bootstrap connection skips WAL.

src/sqtseries/engine/store.py 378 lines

Storage engine: series registry + partitioned measurement storage, with bounded LRU caches.

EXCEPTION

```
SeriesNotFoundError(KeyError)
```

Raised when a metric+tags combination does not exist (resolve with `create_if_missing=False`, or metadata lookup for an unknown `series_id`).

FUNCTION

```
initialize_schema(db, settings=None)
```

Create `series` DDL + indexes, then create the current month's partition and its index if missing. Idempotent — safe to call repeatedly.

CLASS

```
_LRUCache
```

Bounded `OrderedDict` LRU (`get` / `put` / `clear` ; evicts the least-recently-used when over `maxsize`, default 100 000) to cap memory in long runs.

CLASS**StorageEngine**

Fields: `db`, `_series_cache` ((metric, tags_json) → series_id), `_id_cache` (series_id → (metric, tags_json)), `_parts_cache` with a 2 s TTL. Key methods:

- `resolve_series(conn, metric, tags=None, *, create_if_missing=True) -> int` — resolves via the `UNIQUE(metric, tags)` index (verified `SEARCH` plan); inserts the series row when missing and allowed; raises `SeriesNotFoundError` otherwise; caches both directions.
- `get_series_meta(sid) -> (metric, tags_json)` — cached reverse lookup.
- `series_ids_for_metric(metric) -> list[int]` — ordered by `series_id`.
- `series_count() / list_metrics()` — `COUNT(*)` and distinct sorted metrics.
- `insert_many(rows, *, auto_create_partition=True) -> int` — one transaction: resolve every series, validate every partition name, create missing partitions + indexes, then executemany per partition. Returns the number inserted. A failure rolls everything back (no orphan series rows). Nanosecond timestamps.
- `query_time_range(metric=None, series_ids=None, start_ns=None, end_ns=None, *, limit=None, order="asc") -> Iterator[(ts_ns, value)]` — streamed per-partition reads. `series_id` filters use the clustered PK; global limit/order across partitions (partitions iterated newest-first for desc). A partition dropped concurrently (retention) is skipped rather than raising "no such table".
- `first_sample_ts(series_ids, start_ns, end_ns) -> int | None` — oldest sample in range; used by the rollup path's whole-window anchor.
- `_partitions_for_range / _all_partitions / invalidate_partitions` — partition discovery with a 2-second cache; invalidation called after create/drop.
- `close()` — clear caches.

FUNCTION

`_tags_to_json(tags) -> str | None`

orjson with `OPT_SORT_KEYS` (→ stable `UNIQUE(metric, tags)` identity matching `json.dumps(sort_keys=True)`).

src/sqtseries/engine/schema.py 64 lines

Schema DDL and partition-name helpers. Schema (EXPLAIN QUERY PLAN-verified): `series` (metric + JSON tags → integer `series_id`, CHECK `json_valid(tags)`, `UNIQUE(metric, tags)`); `measurements_YYYY_MM` — WITHOUT ROWID, PK (`series_id`, `timestamp_ns`), secondary index on `timestamp_ns`.

`MEASUREMENTS_TABLE_RE = ^measurements_\d{4}_\d{2}$`

`SERIES_DDL` — CREATE TABLE series + unique index.

FUNCTION

`measurements_ddl(year, month) -> str`

CREATE TABLE IF NOT EXISTS for a monthly partition (WITHOUT ROWID + PK).

FUNCTION

`measurements_index_ddl(year, month) -> str`

CREATE INDEX IF NOT EXISTS `ix_measurements_Y_M_ts` on (`timestamp_ns`).

FUNCTION

```
partition_name(year, month) -> str
```

"measurements_YYYY_MM".

FUNCTION

```
month_partition_key(ts_ns) -> str
```

Integer-only math (float division loses precision beyond 2^{53} ns and could round the last nanoseconds of a month into the next — verified: 2025-12-31 23:59:59.999999999 → Jan).

FUNCTION

```
parse_partition_name(name) -> (year, month)
```

Validates against the regex and splits; raises ValueError for anything else. Used to sanitize names before SQL interpolation.

src/sqtseries/engine/pragmas.py 47 lines

PRAGMA helpers. Two classes: one-time file PRAGMAs (page_size, auto_vacuum — fresh file only, before WAL) and per-connection PRAGMAs (WAL etc., applied on every connect).

FUNCTION

```
run_optimize(db)
```

PRAGMA optimize — planner statistics (shutdown/periodic).

FUNCTION

```
wal_checkpoint(db, mode="PASSIVE") -> str
```

Validates mode is PASSIVE/FULL/RESTART/TRUNCATE; returns "busy,log,checkpointed".

FUNCTION

```
quick_check(db) -> str / integrity_check(db) -> list[str]
```

Quick vs full integrity check.

FUNCTION

```
incremental_vacuum(db, pages=100)
```

PRAGMA incremental_vacuum.

FUNCTION

```
run_analyze_once(db)
```

ANALYZE (startup/periodic).

src/sqtseries/engine/migrations.py 70 lines

Lightweight schema migration runner. SQLite ALTER is limited (table rebuild needed for most changes), so migrations are just DDL scripts tracked in a `schema_version` table, applied idempotently at startup. New migrations append to `MIGRATIONS`.

```
MIGRATIONS = [(1, SERIES_DDL)]
```

FUNCTION

```
get_schema_version(db) -> int
```

Ensures the version table exists; returns MAX(version) or 0.

FUNCTION

```
upgrade(db, target=None) -> int
```

Applies pending migrations (each in its own BEGIN IMMEDIATE transaction with an applied_at stamp); returns the new version.

FUNCTION

```
ensure_current_partition(db)
```

Create the current-month partition + index if missing.

FUNCTION

```
run_migrations(db) -> int
```

Upgrade to latest, then ensure the current partition; returns the version.

src/sqtseries/engine/checkpoint.py^{128 lines}

Background WAL checkpoint manager — prevents WAL indefinite growth and detects long-running readers that block checkpoints (a reader holding a read lock prevents TRUNCATE from freeing disk; `PRAGMA wal_checkpoint` reports a nonzero `busy` count).

`_BUSY_WARN_RUNS = 3` — warn after 3 consecutive blocked attempts.

CLASS

```
CheckpointManager
```

Construction: `(db, *, interval=60.0, max_wal_bytes=67108864)`. Stats fields: `wal_bytes`, `last_busy`, `last_result` (busy, log, done), `busy_runs`, `checkpoints`.

- `async start()` — spawns the loop task.
- `async _run()` — sleep interval, then `_checkpoint_if_needed`.
- `async _checkpoint_if_needed()` — stat `db-path-wal`; if $\geq$ `max_wal_bytes` run a TRUNCATE checkpoint (frees disk; PASSIVE does not), record the result, and track consecutive busy runs (log a warning at the threshold, info when unblocked).
- `stats()` — dict for the admin surface.
- `async stop()` — cancel the task, suppressing `CancelledError`.

FUNCTION

```
_parse_result(result) -> (busy, log, done)
```

Parse the "busy,log,checkpointed" string; (0,0,0) on malformed input.

src/sqtseries/engine/backup.py^{127 lines}

Backups via `VACUUM INTO` — a consistent point-in-time snapshot. Caveats (sqlite.org/lang_vacuum.html): the destination must not exist; cannot run inside a transaction; works with any `auto_vacuum` mode.

EXCEPTION

```
BackupExistsError(FileExistsError)
```

Backup destination already exists.

`_PREFIX_RE = ^[A-Za-z0-9_.-]+$` — keeps the prefix safe for filenames and SQL interpolation.

FUNCTION

```
backup_database(db, backup_dir, prefix="sqtseries") -> str
```

Validates the prefix, creates the dir, and runs `VACUUM INTO '<dir>/<prefix>-YYYYmmdd-HHMMSS.db'` on an autocommit connection. Returns the path; raises `BackupExistsError` when the same-second name exists.

FUNCTION

```
backup_latest(db, backup_dir, prefix) -> str | None
```

Newest matching `prefix-*.db` file, or `None`.

FUNCTION

```
_is_fresh(path, interval) -> bool
```

True when the backup's mtime is younger than `interval` seconds.

CLASS

```
BackupManager
```

Background task taking a snapshot every `interval` seconds (default 24 h) in a worker thread (event loop stays responsive on large DBs). First pass runs immediately on boot, so a fresh install gets a backup without stalling startup. A same-second retry (`BackupExistsError`) is counted as a pass and skipped. Fields: `runs` , `backups_created` , `last_backup` . API: `async start()` , `async run_once()` , `async stop()` .

src/sqtseries/engine/maintenance.py 55 lines

Periodic `ANALYZE` so the SQLite query planner's statistics don't go stale as data changes.

CLASS

```
MaintenanceManager
```

`(db, *, interval=3600.0)` . Fields `last_analyze` , `runs` . `async start()` spawns the loop; `async run_once()` runs `ANALYZE` in a thread; the loop sleeps a full interval first (startup already ran `run_analyze_once()`); `async stop()` cancels.

7. Package sqtseries.partition

Retention & partitions: automatic data lifecycle management.

src/sqtseries/partition/__init__.py 25 lines

Exports `PartitionManager` , `RetentionManager` , `RetentionPolicy` , `parse_ttl` , `completed_hour_ns` , `ensure_rollup_table` , `rollup_new_hours` , `rollup_partition` , `rollup_watermark` .

src/sqtseries/partition/manager.py 69 lines

FUNCTION

```
_safe_partition(name)
```

Validates a name against `^measurements_\d{4}_\d{2}$` before SQL interpolation.

CLASS**PartitionManager**

(db, on_change=None) — on_change fired after create/drop (used to invalidate the store's partition cache). Methods:

- list_partitions() -> list[str] — sorted measurements_% tables.
- ensure_partition(year, month) -> str — create table + index if missing (thread-safe via a DDL lock); fires on_change.
- drop_partition(year, month) — DROP TABLE IF EXISTS inside a writer transaction; fires on_change.
- current_partition(now_ns=None) -> str — partition active at the given time.

src/sqtseries/partition/retention.py 180 lines

Retention drops entire monthly partitions (fast DROP TABLE) when the *last day* of a partition falls before the retention horizon. Dropped partitions also have their hours removed from the rollup — deleted data leaves no aggregate trace.

TTL_PATTERN = ^(\d+)([smhdw])\$; UNITS — s/m/h/d/w → timedelta.

FUNCTION**parse_ttl(ttl) -> timedelta**

Parse "30d", "12h", "4w" etc.; ValueError with a helpful message otherwise.

CLASS**RetentionPolicy**

(engine, ttl[, manager]). Methods:

- partitions_to_retain(now=None) -> set[str] — keep partitions whose last day ≥ now - ttl.
- run(now=None) -> list[str] — drop expired partitions (and their rollup rows); returns the dropped names.
- _drop_rollup_rows(year, month) — DELETE from rollup_hourly for that month if the table exists.

CLASS**RetentionManager**

Background task: (engine, store, *, ttl, interval). First pass runs immediately (in a thread) so expired partitions are cleaned on boot. Uses a PartitionManager whose on_change invalidates the store's cache. Fields runs, dropped_total. API: async start(), async run_once(), async stop().

Module helpers: _parse_partition_name, _last_day_of_month, _month_end_dt, _month_start_ns, _month_end_ns (UTC calendar math).

src/sqtseries/partition/rollup.py 280 lines

Rollup pre-aggregation: store per-series per-hour (count, sum, min, max) in rollup_hourly; average is derived as sum/count. Only *completed* hours are rolled; the current hour is always served from raw partitions. The last_hour watermark in rollup_meta records progress; INSERT OR REPLACE keeps re-runs idempotent. Documented assumption: in-order ingestion for completed hours — a write backfilled into an already-rolled past hour is not picked up until a rebuild with replace=True.

Constants

HOURLY_NS = 3_600_000_000_000

ROLLUP_TABLE = "rollup_hourly" / ROLLUP_META = "rollup_meta" / META_LAST_HOUR = "last_hour"

`_PARTITION_RE` — partition-name validator for SQL interpolation.

Functions

FUNCTION

```
completed_hour_ns(now_ns=None) -> int
```

Start of the current in-progress hour (floor to HOUR_NS).

FUNCTION

```
ensure_rollup_table(db)
```

CREATE TABLE IF NOT EXISTS rollup_hourly (WITHOUT ROWID, PK series_id+hour_start_ns) and rollup_meta (k/v).

FUNCTION

```
rollup_watermark(db) -> int
```

Next hour to roll (all earlier hours are rolled); 0 when none.

FUNCTION

```
rollup_partition(db, partition, *, replace=False, cutoff_ns=None) -> int
```

Aggregate a partition's completed hours in `[watermark, cutoff)` into the rollup with one GROUP BY series_id, hour query; `INSERT OR REPLACE`. With `replace=True` the whole month is rebuilt (rollup rows cleared first) for backfill recovery. Returns rows written. The global watermark is advanced by `rollup_new_hours`.

FUNCTION

```
rollup_new_hours(db, now_ns=None, *, skew_s=0.0) -> int
```

Roll every partition's newly-completed hours and advance the watermark. Only hours that ended at least `skew_s` before "now" are rolled — this keeps the fast path exact under the client clock-skew guard (a write arriving up to `skew_s` late can never land in a rolled hour). Idempotent and interruption-safe. Returns rows written.

FUNCTION

```
query_rollup_partial(db, series_ids, start_ns, end_ns, *, bucket_ns) -> list[(bucket, count, sum, min, max)]
```

Partial aggregates from the rollup for the query fast path. `bucket_ns` `None` → one whole-window row (bucket `None`); otherwise one row per epoch-aligned bucket via `(hour_start_ns / ?) * ?`. Bound semantics `[start_ns, end_ns)`; empty list when no rollup rows match.

Private helpers: `_safe_partition`, `_partitions_for_range`, `_month_bounds`.

Class `RollupManager`

CLASS

```
RollupManager
```

`(db, *, interval=300.0, skew_s=0.0)`. Background task: no blocking warm-up on boot — the first pass runs inside the loop task immediately, in a thread, so a large catch-up never stalls startup or the event loop. API: `async start()`, `async run_once()`, `async stop()`.

8. Package `sqtseries.query`

Query engine: aggregation, downsampling, gap filling, and the rollup fast path.

`src/sqtseries/query/__init__.py` 28 lines

Exports `AggregationFunction`, `QueryError`, `TimeSeriesDB`, `aggregate_series`, `downsample`, `downsample_to_intervals`, `gap_fill_linear`, `median`, `p95`, `p99`, `parse_interval`.

`src/sqtseries/query/agg.py` 191 lines

Aggregation, downsampling, and gap-filling functions — pure Python over `(timestamp_ns, value)` samples.

`Sample = tuple[int, float]` — type alias.

ENUM

`AggregationFunction(StrEnum)`

AVG, SUM, MIN, MAX, COUNT, FIRST, LAST, MEDIAN, P95, P99.

FUNCTION

`median(values) / p95(values) / p99(values)`

`statistics.median` and a linear-interpolated percentile (`_percentile`: rank = (n-1)*q, interpolates between the bracketing sorted values). Empty input → NaN.

`AGGREGATORS = {"avg", "sum", "min", "max", "count", "first", "last", "median", "p95", "p99"}` — name → callable.

FUNCTION

`aggregate_series(samples, func) -> float`

Aggregate a whole window. Empty window: count/sum → 0.0, everything else → NaN (never raises `StatisticsError/IndexError`). Unsupported name → `ValueError`.

`INTERVAL_UNITS = {"s": 1, "m": 60, "h": 3600, "d": 86400}`

FUNCTION

`parse_interval(interval) -> int (seconds)`

Parse "5m", "10s", "1h", "1d". Empty/unsupported/<1 → `ValueError`.

FUNCTION

`downsample(samples, interval, func="avg") -> list[Sample]`

Bucket samples by `ts // bucket_ns` and aggregate each bucket; emits only non-empty buckets, epoch-aligned bucket starts.

FUNCTION

`downsample_to_intervals(samples, interval, func="avg", fill_missing=False) -> list[tuple[int, float]]`

Downsample and optionally emit empty buckets (value = NaN) for gaps across the full min → max bucket range.

FUNCTION

`_aggfn(func) -> callable`

Resolve a name or `AggregationFunction`; `ValueError` when unsupported.

FUNCTION

`gap_fill_linear(samples, max_gap_ns) -> list[Sample]`

Linearly interpolate the midpoint between consecutive samples whose gap is `0 < gap ≤ max_gap_ns` (inserts one halfway point per gap). Input must be sorted ascending; larger gaps stay unfilled.

`src/sqtseries/query/builder.py` 417 lines

High-level query API over the storage engine: the spec's `insert / query / query_stream / aggregate / downsample`, with the rollup fast path.

`ROLLUP_FUNCS = frozenset({"avg", "sum", "min", "max", "count"})` — aggregations exactly expressible from the rollup's columns.

EXCEPTION

`QueryError`

Base error for query failures.

CLASS

`TimeSeriesDB`

Wraps a `StorageEngine`. Methods:

- `insert(metric, value, tags=None, timestamp_ns=None) -> int` — single measurement; server time when no timestamp.
- `query(metric=None, series_ids=None, start=None, end=None, *, aggregation=None, interval=None, limit=None, order="asc", fill_gaps_ns=None) -> list[(ts_ns, value)]` — the main entry. If eligible (`_rollup_eligible`) serves from the rollup; otherwise fetches raw and applies: `downsample` (`aggregation+interval`), whole-window aggregate anchored at the oldest sample in the window (matching the rollup path's `first_sample_ts`), or gap-fill. `limit` keeps the first N in the selected order.
- `query_stream(metric, start, end, *, order) -> Iterator` — bounded-memory raw streaming, one series at a time.
- `aggregate(metric, start, end, *, funcs=None) -> dict[str, float]` — per-function whole-window aggregates; each eligible func uses the rollup, and the raw row set is fetched at most once for the others.
- `downsample(metric, start, end, *, interval="5m", aggregation="avg")` — sugar over query.
- `_rollup_eligible(...) -> bool` — requires: aggregation in `ROLLUP_FUNCS`; an interval that is None or a whole-hour multiple; watermark > 0; the window's end hour ≤ watermark; at least one fully-inside hour (span ≥ 2 hours) — otherwise raw is faster; and the start not already past the watermark.
- `_query_rollup(...)` — serve an eligible query: merged per-bucket partials (rollup + raw edges) finalized by `_finalize_buckets`; whole-window form returns a single row anchored at `first_sample_ts`.
- `_rollup_whole_window(...)` — whole-window value for `aggregate()`.
- `_rollup_window_partials(series_ids, start, end, bucket_ns)` — splits `[start, end]` into three disjoint sources so nothing is counted twice: the first (partial) edge hour from raw, the last partial hour from raw, and the fully-inside rolled hours from `query_rollup_partial`; merged per bucket.

Module functions

FUNCTION

`_name(f) -> str`

Normalize an `AggregationFunction` or string to its name.

FUNCTION

`_bucket_partials(samples, bucket_ns) -> list[(bucket, count, sum, min, max)]`

Raw-edge partials matching the rollup's shape; one whole-window partial when `bucket_ns` is None, otherwise one per epoch-aligned bucket (single pass, no sorting needed).

FUNCTION

```
_merge_partials(groups) -> dict[bucket, [count, sum, min, max]]
```

Combine partial aggregates sharing the same bucket key (sums add, min/max take extremes).

FUNCTION

```
_finalize_buckets(merged, name, limit) -> list[(bucket, value)]
```

Emit ascending bucket values via `_value`, honoring limit (first N).

FUNCTION

```
_value(name, cnt, sm, mn, mx) -> float
```

Derive avg (sm/cnt), sum, count, min, max from rollup partials; ValueError for unsupported names.

9. Package sqtseries.messaging

The ZeroMQ layer — externally facing sockets plus internal plumbing. Messages are JSON via orjson; ingest/query/admin use one-part frames, pubsub uses two-part frames `[topic, json-payload]`.

src/sqtseries/messaging/__init__.py 39 lines

Exports `DEFAULT_HWM`, `AdminBroker`, `ConnectionRegistry`, `IngestMessage`, `Ingress`, `ProtocolError`, `PubSub`, `QueryBroker`, `StatsPublisher`, `SubscriptionTracker`, `WorkerPool`, `apply_options`, `dumps`, `new_connection_id`, `parse_admin`, `parse_ingest`, `parse_query`, `socket_options`.

src/sqtseries/messaging/protocol.py 122 lines

Message protocol: framing and validation for ZMQ messages.

EXCEPTION

`ProtocolError(ValueError)`

Malformed or invalid messages. Caught by brokers and mapped to the `INVALID_REQUEST` error shape; in the ingress it increments the invalid counter.

DATACLASS

`IngestMessage`

Validated ingest message: `metric` (str), `value` (float), `tags` (dict|None), `timestamp` (epoch seconds, optional).

Method `to_rows() -> (metric, tags, value, ts_ns)` — defaults to server time, converts seconds → ns.

FUNCTION

`dumps(obj) -> bytes`

orjson with `OPT_NON_STR_KEYS` (non-string keys allowed, e.g. numeric bucket keys).

FUNCTION

`loads(data) -> Any`

orjson parse; JSON errors → `ProtocolError`.

FUNCTION

`parse_ingest(raw, reject_client_timestamp_skew_s=None) -> IngestMessage`

Validates: object body; non-empty string metric; numeric non-bool finite value; tags string → string (keys and values); numeric finite optional timestamp. When `reject_client_timestamp_skew_s > 0` and a timestamp is present, skew beyond the limit raises `ProtocolError` (the clock-skew guard that protects rollup ordering).

FUNCTION

`parse_query(raw) -> dict`

Validates a query: object body, type "query" or omitted, non-empty metric, start/end integers when present; returns the normalized dict.

FUNCTION

`parse_admin(raw) -> str`

Validates an admin message and returns the command string.

src/sqtseries/messaging/context.py 59 lines

ZeroMQ context and socket helpers. Benchmark-derived defaults: LINGER must be set BEFORE bind/connect (pyzmq#1407); server sockets LINGER 500 ms (clean shutdown drain); client sockets 0 ms; HWM 10 000 for high-traffic ingestion.

```
DEFAULT_HWM = 10000 / SERVER_LINGER_MS = 500 / CLIENT_LINGER_MS = 0
```

FUNCTION

```
socket_options(*, hwm=10000, linger_ms=500, sndhwm=None, rcvhwm=None, max_msg_size=50 MiB, heartbeat_ivl=1000, heartbeat_timeout=5000, heartbeat_ttl=5000, immediate=True) -> dict
```

Returns the option dict to apply before bind/connect: LINGER, MAXMSGSIZE, HEARTBEAT_*, IMMEDIATE, plus SNDHWM/RCVHWM.

FUNCTION

```
apply_options(sock, options) -> sock
```

Applies options, logging (and tolerating) per-option failures — defensive.

src/sqtseries/messaging/ingress.py^{136 lines}

Ingress pipeline: ZMQ PULL → validate → sink (SQLite write) + republish. No app-level write queue — each validated frame goes straight to the sink and onto the pub/sub bus.

```
TOPIC_PREFIX = b"" — pubsub topic is the metric name.
```

CLASS

Ingress

(endpoint, settings, sink=None, on_publish=None, context=None). `sink(metric, tags, value, ts_ns)` is called synchronously per frame (None skips persistence — validation and republish still happen);

`on_publish(bytes_metric, dict)` feeds live subscribers. Counters: `recv_count`, `error_count`, `invalid_count`.

- `async start()` — PULL socket with HWM + max message size, bound to the endpoint.
- `async run_once(block=True) -> bool` — recv one frame (blocking or NOBLOCK), handle it, return whether one was handled; zmq errors counted and logged.
- `async drain()` — drain ALL pending frames in a tight loop (shutdown/tests).
- `_handle(raw)` — parse (invalid JSON counts), `parse_ingest` with the skew guard (ProtocolError/OverflowError → invalid counter — OverflowError guards the huge-float timestamp path where `ts * 1e9` can't fit an int), then sink + `on_publish`.
- `async stop()` — close the socket (linger 0).
- `stats()` — {recv, invalid, errors}.

src/sqtseries/messaging/broker.py^{204 lines}

Query broker: REP (or ROUTER) socket serving query/aggregate requests with a pluggable handler and an optional handler timeout.

CLASS

QueryBroker

(endpoint, settings, handler=None, context=None, use_router=False, handler_timeout_s=None). Counters `requests`, `errors`.

- `async start()` — REP (default) or ROUTER socket, HWM 1000, bound.
- `async run_once(block=True) -> bool` — one request: router multipart or plain frame; returns whether handled.
- `_handle(raw) / _handle_router(frames)` — parse (ProtocolError → INVALID_REQUEST), dispatch, and reply; TimeoutError → QUERY_TIMEOUT; any other exception → INTERNAL_ERROR (logged). Replies on REP are sent *blocking* — a REP socket requires strict recv/send alternation, so dropping a reply on EAGAIN would wedge the broker for every client; router replies use NOBLOCK with a warning on would-block.
- `async _dispatch(query)` — with `handler_timeout_s` the blocking handler runs off the loop in a thread under `asyncio.wait_for` (a slow query never stalls the event loop; over budget → QUERY_TIMEOUT). None/≤0 runs synchronously inline.
- `async stop()`; `stats()` — {requests, errors}.

src/sqtseries/messaging/admin.py 19 lines

class AdminBroker(QueryBroker) — REP socket for admin commands on port 12504; messages are `{"cmd": "..."}.` Heavy operations (backup/vacuum) run in the handler and therefore pause ingestion for their duration; the offline CLI (`sqtseries vacuum`) is the recommended path for large maintenance.

src/sqtseries/messaging/pubsub.py 174 lines

PUB/SUB live-subscription bus with subscriber tracking via XPUB: the service receives subscribe/unsubscribe events directly from the wire — no polling.

CLASS

SubscriptionTracker

Tracks active vs *lingering* topics. `subscribe(topic)` activates; `unsubscribe(topic)` moves it to `_linger_until` for `linger_seconds` (a ZMQ subscriber disconnect/reconnect window); `cleanup()` sweeps expired linger entries; properties `active_topics`, `stats()` ({active, lingering}).

CLASS

PubSub

(endpoint, *, linger_seconds=30.0, hwm=10000, context=None, registry=None).

- `async start()` — XPUB socket with `XPUB_VERBOSE=1` (emit an event for EVERY join/leave, not just topic-trie transitions — without it a second subscriber to a known topic generates no event and counts would be wrong), RCVHWM 0 (unlimited inbound subscriptions), LINGER 500, SNDHWM hwm; plus a background reader task.
- `async publish(topic, payload)` — send `[topic, json]`; increments `published` (publish attempts, not deliveries — libzmq may drop frames for slow subscribers).
- `_read_subscriptions()` — decode `\x01topic` (subscribe) and `\x00topic` (unsubscribe) events; update the tracker and the optional registry (register/unregister_zmq_sub) in real time; idle ticks sweep expired linger topics.
- `async stop()`; `stats()` — {published, active, lingering}.

src/sqtseries/messaging/connection_registry.py 190 lines

Connection and subscription registry: tracks every active WebSocket connection and ZMQ SUB subscriber so the service can answer `connections`, `conncheck`, and `subscribers` queries at any time. Events carry arrival/leaving timestamps (`connected_at` / `left_at` for WS; `arrived_at` / `left_at` plus `first_seen` for ZMQ topics).

CLASS

ConnectionRegistry

Internal maps: `_ws` (`conn_id` → entry), `_zmq_subs` (topic → count), `_zmq_first_seen`, `_listeners`. Methods:

- `on_event(callback) / remove_listener(callback)` — subscribe/unsubscribe to (`event_type`, `payload`) pairs ("conn", "sub"). `remove_listener` must be paired with every `on_event` or repeated start/stop cycles grow listeners without bound.
- `register_ws(conn_id, peer, topic)` — record a WS join; emit a "conn" event (`connected: True`, ttl 60).
- `unregister_ws(conn_id)` — mark gone; the leave event carries `connected_at` and `left_at` for duration display.
- `register_zmq_sub(topic) / unregister_zmq_sub(topic)` — increment/decrement per-topic counts, emit "sub" events; `first_seen` set when a topic goes 0 → 1 (ttl 30 on joins).
- `check_connection(conn_id) -> bool` — `conncheck` support.
- `list_connections() -> list[dict]` — sorted by `connected_at`.
- `subscriber_count(topic=None) -> int`, `active_topics()`, properties `ws_count` / `zmq_sub_count`.
- `snapshot() -> dict` — read-only summary: `ws_connections`, `zmq_subscribers`, `active_topics`, `connections` list, per-topic subscriptions with `first_seen` (used by `/ws/connections`, admin subscribers, and the stats report).

FUNCTION

`new_connection_id() -> str`

`uuid4().hex[:12]` — short connection IDs.

src/sqtseries/messaging/stats_publisher.py^{108 lines}

Stats PUB socket (12506): broadcasts connection and subscription events in JSON; consumers subscribe with prefix filters (`conn`, `sub`, `report`, or `""`).

`REPORT_INTERVAL = 10.0`

CLASS

StatsPublisher

(`endpoint`, `registry`, `*`, `report_interval=10.0`, `context=None`).

- `async start()` — PUB socket (LINGER 0, SNDHWM 1000) bound; starts the report loop; hooks the registry via `on_event`.
- `async publish(event_type, payload)` — one frame, orjson with default=str.
- `_on_registry_event(...)` — forwards registry events into the socket via tracked tasks (drained on stop).
- `_report_loop()` — every interval publishes a "report" event: `uptime_s`, `ws_connections`, `zmq_subscribers`, `active_topics`.
- `async stop()` — unhooks the listener FIRST (no new tasks), cancels the loop, drains in-flight forwards, closes the socket.

src/sqtseries/messaging/worker.py^{74 lines}

Worker pool: N async workers consuming from ingress and serving queries. ZMQ sockets are not thread-shared; async tasks on the same socket are safe only via the single event loop — so the pool lives on one loop with cooperative workers. (For multiple processes: ProcessPool + per-process context.)

CLASS

WorkerPool

`(step, *, size=1)` where `step()` is an async callable doing one unit of work and returning True if it did any. `async start()` spawns `size` tasks; each worker loop calls step, sleeps **0 ms when work flowed, 10 ms when idle** — a NOBLOCK recv raising zmq.Again does NOT release the event loop, so a tight idle loop would starve timers and CPU-bound threads (verified: 112k spins/sec starved call_later; 10 ms only delays the first message after idle, and busy cycles use sleep(0) with no added latency). `async stop()` cancels and awaits all; property `running`.

10. Package sqtseries.gateway

HTTP gateway (FastAPI REST + WebSocket) on port 12505. Bridges HTTP directly to the engine — single-process architecture, no ZMQ hop.

src/sqtseries/gateway/__init__.py 6 lines

Exports `create_app` and `router`.

src/sqtseries/gateway/app.py 218 lines

REST + WebSocket gateway factory with CORS, rate limiting, request-ID and security-header middleware.

FUNCTION

```
create_app(store=None, tsdb=None, settings=None, pubsub=None, ingestion=None, registry=None, query_timeout_s=None) -> FastAPI
```

Builds the app. `tsdb` is created from `store` when absent. Everything shared lives on `app.state`: `tsdb`, `store`, `pubsub`, `settings`, `ingestion` (skew guard for HTTP writes), `registry`, `query_timeout_s`, `ws_count` / `ws_max_connections` (live WebSocket budget). Middleware (LIFO — last added runs outermost): request-ID middleware (outermost; echoes or generates an `X-Request-ID`), security headers middleware (CSP `default-src 'self'`, X-Frame-Options DENY, nosniff, X-XSS-Protection, Referrer-Policy, Permissions-Policy), rate limiter, then CORS nearest the router. CORS and the rate limiter short-circuit non-HTTP scopes, so WebSockets bypass both. Endpoints:

- `GET /api/v1/health` — {"status": ok|degraded, "version"} via `is_ready`.
- `WS /ws/subscribe?metric=*` — accept; enforce the 1000-connection cap (close 1013 "too many connections"); delegate to `subscribe_and_forward`; close 1008 when streaming is disabled; decrement the slot in finally.
- `WS /ws/connections` — push-only live connection/subscription feed: sends a snapshot on connect, then one frame per registry change. Registry callbacks may fire from any thread, so events are marshalled onto the loop via `call_soon_threadsafe` into a bounded queue (backlog > 500 drops the oldest — a stalled client must not grow memory unbounded), drained by a sender task paired with a disconnect watcher.

FUNCTION

```
_ws_slot_available(app) -> bool
```

Check-and-increment of the WebSocket cap. Single-threaded event loop: no await between check and increment, so concurrent handlers cannot overshoot the cap.

src/sqtseries/gateway/routes.py 292 lines

REST routes, all under `/api/v1`. Times on the wire are epoch *seconds*; the engine uses ns internally.

`_WRITE_BODY = Body(...)` — module-level sentinel for FastAPI body binding; `_DEFAULT_QUERY_TIMEOUT_S = 30.0`.

POST

`/api/v1/write`

Ingest one object or an array of rows. Batch semantics: validate the WHOLE batch first (a bad item rejects the batch — retry-safe, nothing partially persists), persist in ONE transaction (`_insert_validated_batch`; timestamp-less rows get distinct ns values so two same-series rows can't collide on the clustered PK), then republish to live subscribers (`_broadcast` — best-effort; a failing broadcast must not turn a successful write into a 500). Returns {"status": "ok", "written": n}. Validation mirrors the ZMQ path: non-empty metric, finite number, string → string tags, numeric timestamp within int64 ns and within the clock skew guard.

GET

```
/api/v1/read?metric=&start=&end=&aggregation=&interval=&limit=&order=
```

Query rows; returns {"status": "ok", "data": [{timestamp (s), value}]} limit ge=1 le=100000, order pattern ^(asc|desc)\$. Runs off the event loop in a thread under query_timeout_s (504 GATEWAY_TIMEOUT on expiry).

GET

```
/api/v1/aggregate?metric=&start=&end=&funcs=
```

Comma-separated funcs (default "avg") over a window; returns {"status": "ok", "aggregations": {func: value}}.

GET

```
/api/v1/stats
```

{"status": "ok", "metrics": n, "series": n} from the store.

GET

```
/api/v1/connections
```

Live WebSocket connection list from the registry (kind/peer/topic/connected_at).

GET

```
/api/v1/subscribers
```

Live ZMQ subscriptions: {"status": "ok", "zmq_subscribers": n, "subscriptions": [...]}.

Helpers**FUNCTION**

```
_to_ns(seconds) -> int | None
```

Seconds → ns; overflow raises 400.

FUNCTION

```
_validate_row(row, max_skew) -> (metric, value, tags, ts_ns)
```

Full per-row validation with HTTPException(400) details; rejects timestamps outside int64 ns and beyond the skew guard.

FUNCTION

```
_broadcast(pubsub, rows)
```

Best-effort republish of HTTP writes.

FUNCTION

```
_insert_validated / _insert_validated_batch
```

Single-row insert / one-transaction batch insert with distinct server ns for timestamp-less rows.

FUNCTION

```
async _run_query(request, fn)
```

Run a blocking query off the loop (asyncio.to_thread + wait_for, timeout from query.timeout_s) so a slow aggregate can't stall WS streaming or admin; abandoned threads finish in the background and their results are discarded.

src/sqtseries/gateway/ratelimit.py 63 lines

Fixed-window rate limiting middleware.

CLASS`RateLimitMiddleware`

ASGI middleware rejecting requests over `limit_per_minute` per fixed 60 s window, keyed by the client peer address. The X-Forwarded-For header is deliberately NOT trusted (client-controlled; anyone could rotate it to bypass). Memory-bounded: keys from other windows are swept when the table exceeds 10 000 entries, and excess current-window keys are evicted FIFO. Over-limit replies with the standard error shape and HTTP 429.

`src/sqtseries/gateway/websocket.py` 115 lines

WebSocket subscription handler — proxies live measurements from the pubsub XPUB socket to the WS client and tracks connection state in the registry. The browser dashboard opens `ws://host:12505/ws/subscribe?metric=...` and receives every published measurement.

`SEND_TIMEOUT = 30.0` — a frame that can't be flushed within 30 s means the client is dead; close it (1011 slow consumer) instead of holding the connection and its ZMQ subscription hostage.

FUNCTION`async subscribe_and_forward(websocket, *, pubsub, topic="", registry=None)`

Creates a `conn_id` and registers the WS (peer host:port, topic). Opens a SUB socket and subscribes *before* connect (the subscription then travels with the handshake; subscribing after connect leaves a race where a measurement published in between is dropped). Two sibling tasks: `send_loop` (recv_multipart with a 30 s wait_for timeout; on timeout sends a `{"type": "ping"}` keepalive; filters by topic prefix; decodes payload with errors="replace") and `watch_disconnect` (consumes inbound frames so a client disconnect surfaces immediately instead of lingering until the next send — which would hold uvicorn's graceful shutdown). First task to finish kills the other; the registry entry is always unregistered in finally and the socket closed.

FUNCTION`async _send(websocket, text, what) -> bool`

`send_text` under `SEND_TIMEOUT`; on timeout logs, closes 1011, returns False so the caller aborts the stream.

11. Examples

Standalone runnable scripts for every operation. None of them import sqtseries internals (the Client is used where convenient) — the wire-protocol ones replicate what any language can do. Running them requires a live `sqtseries run` service on the default ports (except `run_custom.py` which starts its own).

examples/producer.py^{79 lines}

Send measurements via raw ZMQ PUSH to port 12501 — no Client, pure pyzmq: the fastest write path, and a template for any language with a ZMQ binding.

- `main()` — args: `--host`, `--port` (12501), `--rate` (msg/s; 0 = as fast as possible), `--count` (0 = forever), `--batch` (checkpoint print every N). Builds a PUSH socket (SNDHWM 10000, LINGER 500), picks random metric/host from small pools, and sends one JSON frame per point. Every `batch` points it prints throughput. Closes the socket with `linger=2000` so queued points flush instead of being dropped (fire-and-forget PUSH loses everything with `linger 0`).

examples/consumer.py^{71 lines}

Query sqtseries from a separate process via ZMQ REQ or HTTP — also without importing sqtseries.

- `main()` — `--port` (12502) or `--http <base>`; `--host`.
- `zmq_query(host, port)` — REQ socket; sends `{"type": "query", "metric": "demo.cpu", "aggregation": "avg", "interval": "1m"}`; prints the pretty reply.
- `http_query(base)` — GET `/api/v1/read?metric=demo.cpu&aggregation=avg&interval=1m` via `urllib`.

examples/stats_monitor.py^{81 lines}

Live activity monitor for the stats PUB socket (12506): prints every connect, disconnect, and subscription change with a timestamp — a template for dashboards or alerting pipelines.

- `main()` — positional `filter` (conn | sub | report | empty = all); SUB socket with the prefix filter. Prints formatted lines: `CONNECTED/DISCONNECTED` (kind, id, peer), `SUB` (topic, listeners), `REPORT` (ws count, `zmq_subs`, topics, uptime).

examples/ingest_http.py^{87 lines}

Send measurements over plain HTTP — works with curl, urllib, fetch, any language. Use when you can't or prefer not to install a ZMQ binding.

- `main()` — args: `--host`, `--port` (12505), `--rate`, `--batch` (points per request), `--count`. Builds a batch of random sensor readings (temp/humidity/pressure with a room tag), POSTs the array to `/api/v1/write`, prints per-request written counts. Demonstrates the batch endpoint's atomicity (validate-then-persist in one transaction).

examples/admin_examples.py^{127 lines}

Exercises all nine admin commands against a running service and prints their replies.

- `admin(endpoint, cmd, **kwargs)` -> dict — one REQ round trip with `{"cmd": ..., **kwargs}`.
- `main()` — walks through **ping** (asserts pong), **health** (status/version/uptime/database), **stats** (prints the key counters), **connections**, **conncheck** (checks all live ids plus a fake one), **subscribers**, **optimize**, **backup**, **vacuum** (notes that VACUUM while running may fail — the offline path is `sqtseries stop && sqtseries vacuum`).

examples/stream_examples.py^{128 lines}

Subscribe to live measurement streams three ways: raw ZMQ SUB (12503), the Python Client, and WebSocket (12505). Every accepted measurement is republished to matching topic prefixes.

- `subscribe_zmq(host, port, topic)` — SUB socket, prefix filter, prints `[topic] metric=value tags`.
- `subscribe_client(host, topic, port)` — `Client.subscribe` generator; skips the None heartbeat yield.
- `subscribe_websocket(host, topic, http_port)` — `websockets` library; connects to `/ws/subscribe?metric=...`; ignores the `{"type": "ping"}` keepalive frames and prints idle pings.
- `main()` — `topic` positional (* = all), `--ws`, `--client`, `--port`, `--http-port`.

examples/tags_examples.py^{157 lines}

A runnable walkthrough of tags: what they are, why to use them, and how to query per-tag (the wire queries whole metrics; per-tag breakdowns use the embedded API with `--db`).

- `seed(c)` — 6 series: `cpu.usage` for `web1/web2/web3` with `{host, env}`; `temp.celsius` for `lobby/kitchen/basement` with `{room, floor}`.
- `whole_metric(c)` — merged aggregates over the wire (avg/max/count, min/max).
- `per_tag(db)` — embedded: for each metric, group `series_ids` by a tag value via `store.series_ids_for_metric` + `get_series_meta`, then query each group's last 24 h and print point counts + averages per tag.
- `show_validation(http_port)` — demonstrates that non-string tag values (`{"cpu_count": 8}`) get a clear HTTP 400.
- `main()` — prints the six-part catalog including the cardinality warning: tag dimensions you slice by, never unique IDs, and put the reading in "value".

examples/run_custom.py^{162 lines}

Runs your own instance with custom ports and a fresh database — exactly the production flow: write a config, start with `-config`, verify, stop. No `sqtseries` code drives the lifecycle (only the `Client` for write/query).

- `free_port()` — OS-assigned free port.
- `write_config(workdir) -> ports` — writes `config.toml` with six distinct free ports and `auto_detect = false`.
- `wait_until_ping(port, timeout_s=20.0)` — polls the admin REP socket until `{"cmd": "ping"}` answers pong.
- `main()` — `--workdir` (default `sqt-custom-work/`); writes the config, spawns `python3 -m sqtseries --config ... run` as a background process, waits for ping, writes + queries one metric via the `Client` on the custom ports, then stops via the `stop` command (which reads `runtime.json`). Falls back to SIGTERM if the stop command doesn't finish.

examples/maintenance_examples.py^{169 lines}

Maintenance operations and health checks: backup, vacuum, optimize, and a configurable health-check loop for alerting.

- `admin(host, cmd, port, **kwargs)` — REQ helper.
- `health_check(host, admin_port, http_port) -> bool` — checks ZMQ admin "health" and the HTTP `/api/v1/health` endpoint.
- `backup_database` / `optimize_planner` / `vacuum_database` / `show_stats` — the four maintenance operations with friendly output; vacuum prints the offline recipe when it fails while running.
- `main()` — `--backup-only`, `--health-loop N` (alert loop), `--admin-port` / `--http-port` overrides.

examples/query_examples.py^{225 lines}

Every query type demonstrated: raw reads, limit/order, all ten whole-window aggregations, downsampling, multi-aggregation, HTTP read and aggregate. Seeds 120 deterministic `demo.cpu` points first so results are real.

- `zmq_req(endpoint, payload)` — REQ helper.
- `seed_data(host, port)` — 120 points at 30 s intervals over the last hour; closes with `linger=2000` so nothing is dropped.
- Query helpers: `raw_query`, `raw_query_limit`, `whole_window_agg` (loops avg → p99), `downsample_query` (1m buckets), `multi_agg` (one request, several funcs), `http_query`, `http_aggregate`.
- Calls out that gap filling is embedded-API only (no wire parameter).

examples/question_examples.py^{412 lines}

The question catalog: 13 real questions people ask a time-series database, each answered with working code against one running service. Seed data is deterministic so printed answers are reproducible; every `ANSWER:` line is machine-readable (key = value) so the test suite can assert on it.

Prerequisite: the demo writes "history" (yesterday, last week), so the default clock-skew guard (300 s) would reject it. Start the service with `reject_client_timestamp_skew_s = 864000` (10 days), as documented at the top of the file.

- `_fmt(v, spec)` — format or "no data" for None/NaN.
- Time helpers: `hour_ago_ns()`, `day_start_ns()`, `yesterday_start_ns()`, `week_ago_ns()`.
- class `ExampleEnv` — one live instance: `seed()` writes deterministic series (cpu.load 6 h × 5 min; temp.celsius 24 h × 60 min sine wave; latency.web and visitors.web 8 days × 30 min with deterministic cycles); `send(metric, value, ts_s)` writes with explicit timestamps. Client built with subscribe/admin ports disabled (-1).
- Answers: `answer_avg_cpu_last_hour`, `answer_peak_temp_yesterday`, `answer_p99_week`, `answer_count_today` — plus extras: `extra_trend` (this hour vs previous), `extra_per_minute_curve` (downsampling), `extra_hourly_buckets` / `extra_daily_buckets` (min/avg/max per bucket), `extra_busiest` (busiest hour/day by visitor sum), `extra_visitors_sum`, `extra_staleness` (age of the newest reading via limit=1 order=desc), `extra_alltime` (no window + rolling 3-day stats ladder), `http_variants` (the same four questions as curl-able HTTP URLs).

examples/question_examples_advanced.py^{454 lines}

The next-level catalog: 10 questions needing per-series breakdowns, two windows at once, or arithmetic on top of results. `--db PATH` unlocks questions 1–3 which read series directly from the database file (the wire protocol queries a whole metric); everything else runs over ZMQ/HTTP. Seeds ~13k deterministic points (hosts, pages, visitors with a daily rhythm and growth, cpu.load).

- `utc_day_start(days_ago)` — UTC midnight in epoch seconds.
- class `ExampleEnv` — extended with `wait_for_ingest(expected, timeout)` (polls admin stats until the service reports the ingested count — ZMQ ingestion is fire-and-forget, so this makes the demo deterministic on any machine; the `send` helper also sleeps 50 ms every 2000 points to keep the client HWM from filling), and embedded-read helpers `open_store()`, `series_ids(metric)`, `series_tags(sid)`, `series_aggregate(sid, start_s, end_s, func)`.
- Questions: `q1_busiest_host` (per-series avg → ranking), `q2_p95_per_page`, `q3_visitors_per_page`, `q4_throughput` (count/60 per minute), `q5_percentile_ladder` (median/p95/p99), `q6_compare_days` (today vs yesterday vs a week ago), `q7_busiest_hour` (sum & 1h buckets → max), `q8_data_quality` (expected vs received readings today), `q9_trend` (day-over-day count slope), `q10_staleness`.

12. Camera Analytics Stack (examples/camera)

A complete self-contained demo: a *simulated camera* with edge analytics streams JSON over a WebSocket; `camera_feed.py` pumps every reading into sqtseries over plain HTTP (no sqtseries import needed — works from any Python environment); `camera_feed.py --ask` answers 17 questions about the accumulated data; `overlay.html` is a live dashboard that also subscribes to sqtseries and to the connection feed; `sqtcam.sh` starts everything in a tmux session.

examples/camera/overlay_server.py^{235 lines}

Overlay HTTP server + WebSocket data generator (FastAPI + uvicorn). Serves `overlay.html` on HTTP and simulated analytics on `/ws/metrics` — every 0.25 s by default. Used by OBS browser sources (hence 0.0.0.0 and the S104 ruff ignore).

- `CONFIG` — host from `TEST_HOST`, port from `--port /TEST_PORT/` random, `push_interval 0.25`, `error_interval 0` (simulated errors off), `camera_id test_cam_001`.
- `free_port()` — OS-assigned free port.
- class `DataSimulator` — stateful random generator: visible people/cars (random-walk around bases), monotonic totals, per-minute rates, battery percent drifting down, battery/cpu temperature, cpu usage; every message carries

`generated_at` (ISO-8601 UTC) and `system_metrics`. `should_error()` optionally injects an error payload every `error_interval` seconds.

- class `ConnectionManager` — accept/track/broadcast WebSockets; on broadcast, dead sockets are detected via `return_exceptions` and disconnected.
- Routes: GET `/overlay.html` and GET `/` serve the dashboard (single self-contained file by design — a trusted, developer-owned artifact for LAN/OBS use; the real API surface, the `sqtseries` gateway, sends strict security headers); WS `/ws/metrics` — broadcast a payload every `push_interval` using `wait_for(receive_text, timeout)` as the pacing timer.
- `main(argv)` — resolves the port and runs `uvicorn` with `ws_max_size` 1 MiB, `ws_ping` 25/10.

examples/camera/camera_feed.py^{693 lines} — the biggest example

Two parts: **Pump** (default) subscribes to a metrics WebSocket and writes every reading to `sqtseries` via HTTP; **Ask** (`--ask`) answers 17 questions from the stored data. Only needs `stdlib` + `websockets` (pump mode) — never imports `sqtseries`.

Flattening (the core concept)

Each camera message is flattened into flat `metric + value + tags` measurements tagged with `camera_id`. 10 numeric fields (FIELD_PATHS, dotted JSON paths like `system_metrics.batteryPercent`) plus 1 boolean (BOOL_PATHS: `detection_overlay_enabled` → 1.0/0.0). String fields (`batteryHealth`, `footer`, `overlay URLs`) can't be stored as numbers and are skipped.

- `agg(base, metric, funcs, start_s, end_s)` — `/api/v1/aggregate` helper.
- `series(base, metric, start_s, end_s, interval, func)` — `/api/v1/read` with downsampling; returns `[(bucket_ts_s, value)]`.
- `latest(base, metric, start_s)` — newest reading (`limit=1, desc`).
- `flatten(msg)` → `list[dict]` — one WebSocket message → write points; uses the message's `generated_at` as the timestamp (`_source_ts`), falls back to server time.
- `post_batch(base_url, points)` — POST the array; prints HTTP errors (0 for unreachable).
- The 17 questions: `q_traffic_current` (averages & extremes), `q_busiest_hour_day`, `q_footfall_total`, `q_trend` (day-over-day), `q_turnover` (total vs visible), `q_percentiles` (capacity planning), `q_anomaly` (current hour vs same hour previous days: mean + 2σ threshold), `q_battery` (drain rate and projected time-to-empty), `q_cpu` (usage/temperature profile + >70 °C warning), `q_correlation` (Pearson r footfall vs CPU per hour), `q_battery_vs_busy`, `q_staleness`, `q_uptime` (received vs expected readings, ~4/s), `q_multicam` (notes the wire API can't split per-camera — points to `--db`), `q_monthly_projection`, `q_storage` (points/day → MB/day at ~100 B/pt), `q_overlay` (enabled % of time).
- `ask(base, hours)` — runs ALL_QUESTIONS over `[now - hours, now]`, each guarded by try/except so one bad metric can't kill the catalog.
- `main()` — `--ws URL`, `--host / --http-port`, `--ask`, `--hours`, `--once` (pump one message and exit, for tests). Pump mode: subscribe → flatten → post each message.

examples/camera/overlay.html^{821 lines} — single-file dashboard

The live camera dashboard: a single self-contained HTML page (inline CSS + JS, GitHub-dark theme) meant for OBS browser sources. It connects to `/ws/metrics` for camera telemetry and — when opened with `?sqtseries=ws://<host>:<port>/ws/subscribe` — also subscribes to `sqtseries` itself and to the `/ws/connections` feed, so the dashboard doubles as a demonstration of the time-series database's live path.

Layout

Header (camera id, connection-state badge with animated dot, last-updated), an error banner for simulated feed errors, and cards: People (visible now hero, detected total, per minute), Vehicles, Battery (charge hero, temperature, health), CPU (usage hero, temperature, last-reading age), Detection overlay (status, stream URL, footer), and a large *Time-series DB* card: connection badge, frame counter, live connected-clients table, ZMQ subscribers table, and an activity log. A collapsible raw-last-message viewer and a footer complete the page.

JavaScript behavior

- `render(msg)` — updates every metric from one camera message; `refreshAge()` re-renders the "last reading" age every second.
- `connect()` — camera WS with exponential backoff reconnect (1 → 10 s cap); error payloads → banner.
- `connectDb()` — sqtseries WS subscription; records the latest value per `metric|camera_id` key into a 15-entry Map; ignores `{"type": "ping"}` keepalives; renders the db table with per-entry age.
- `connectConnWs()` — `/ws/connections` feed: applies the initial snapshot and then `conn`/`sub` events (`applyConnEvent` / `applySubEvent`) to two Maps; `renderClients()` renders both tables; activity lines are appended (bounded to 60).
- Security posture: inline CSS/JS is intentional for this trusted developer-owned artifact (no user input, no third-party content); the sqtseries gateway itself sends strict headers.

examples/camera/conn_validate.py^{238 lines}

Validates the live connection/subscription feed of a running sqtseries. Connects N WebSocket subscribers and M ZMQ SUB subscribers, watches the push-only `/ws/connections` feed, and asserts every join and leave arrives instantly with the exact figures. Exits non-zero on any mismatch. This backs the camera dashboard's "Connected clients" / "ZMQ subscribers" tables.

- `next_msg(ws, timeout)` — one JSON frame with a timeout.
- `wait_for(ws, pred, timeout, label)` — consume frames until pred; AssertionError with everything seen on timeout.
- `validate(args)` — the full scenario:
 1. Connect to `/ws/connections`; record the baseline snapshot (ids, per-topic zmq counts).
 2. Join `--clients` WebSocket subscribers on distinct topics (`v0.` ...) — assert a matching `conn` event within timeout, with `connected_at`.
 3. Join `--zmq Subs` ZMQ subscribers on distinct topics (`cpu0.` ...) — assert `sub` events with `arrived_at`, and exact per-topic counts.
 4. Cross-check via HTTP `/api/v1/connections` and `/api/v1/subscribers` — exact figures beyond the baseline.
 5. Leave WS clients one at a time (reverse order) — assert leave events with `connected_at` ≤ `left_at`.
 6. Leave ZMQ subscribers — assert counts return to 0, with `left_at` ≥ `first_seen`.
 7. Final HTTP check: back to the baseline exactly.
- `parse_args / main(argv)` — CLI with `--clients` (3), `--zmq Subs` (2), `--timeout` (5 s).

examples/camera/sqtcam.sh^{102 lines}

One command to run the whole demo: starts sqtseries, the overlay simulator, and the camera feed in a tmux session (`sqtcam`) so data can collect overnight.

- Uses a dedicated work dir `/tmp/sqtcam` with its own config (ports 17101–17106, `auto_detect` off) — never touches the default service.
- Prefers the project venv python (`~/ .env/sqtseries/bin/python3`), falls back to `python3`.
- Window 1: `python3 -m sqtseries --config conf.toml run`. Window 2: the overlay server, logging to `overlay.log`. It then greps the log for the overlay's reported WebSocket port and starts window 3: `camera_feed.py --ws ws://localhost:PORT/ws/metrics --http-port 17105`.
- Re-running prints attach instructions instead of duplicating the session.

13. scripts/benchmark.py

scripts/benchmark.py^{182 lines}

Reproducible performance benchmark: SQLite insert throughput + query latency, measuring the raw engine and the rollout fast path. Data is written at evenly spaced intervals over `--span-hours` across `--series` series, ending on a completed hour boundary two hours in the past so every measurement is rollable.

- `_completed_hour_end()` — start of a completed hour two hours back.
- `_data_bounds(engine)` — global MIN/MAX timestamp across partitions.
- `bench_inserts(db_path, rows, batch, series, span_hours) -> dict` — inserts rows in batches via `store.insert_many`, tags `{host, id}`; returns rows/seconds/rows_per_sec.
- `run_queries(db_path, n) -> dict` — measures p50/p95 latencies for: (1) raw single-series 1 h range (clustered PK path); (2) the wide-window avg/1h query before rollup (raw path); (3) the same query after `rollup_new_hours` — and reports the `speedup` ratio.
- `main()` — `--db`, `--rows` (200k), `--batch` (5000), `--series` (1000), `--span-hours` (24), `--queries` (100).

14. Tests (54 files + conftest.py)

The suite is self-contained: every test uses a temp database under `tmp_path` — no pre-existing data or running service required. Service-spawning tests use OS-assigned free ports (`free_port()`), so nothing clashes with a local instance. `asyncio_mode = "auto"` awaits async tests without markers; a 60 s pytest-timeout guards hangs. Run: `python3 -m pytest tests/ -q`.

tests/conftest.py^{85 lines}

- `free_port()` — OS-assigned free port (bind port 0).
- `@pytest.fixture free_ports` — six distinct free ports for one service instance (ingest/query/streaming/admin/http/stats).
- `@pytest.fixture(autouse=True) _silence_sqtseries_logs` — routes "sqtseries" stdlib loggers to CRITICAL so background tasks don't flood test output.
- `@pytest.fixture tmp_db_path` — `tmp_path/data/test.sqlite`.
- `@pytest.fixture sample_toml / sample_json` — sample config files with ports in the 14xxx range.

tests/ — every module with its test functions^{~8.5k lines}

Grouped by area (per tests/README.md and the code). Test names below each module summarize the covered behaviors.

Engine: schema, storage, PRAGMAs, durability

File	Test functions / coverage
<code>test_engine.py</code>	<code>tables_created</code> , <code>series_tables_without_rowid</code> , <code>secondary_timestamp_index</code> , <code>unique_series_index</code> , <code>resolve_creates_new</code> , <code>resolve_not_found</code> , <code>resolve_deduplicates</code> , <code>series_meta</code> , <code>insert_many_returns_count</code> , <code>insert_empty</code> , <code>insert_bulk_partitions</code> , <code>range_query(+desc)</code> , <code>range_query_missing_metric</code> , <code>range_query_requires_arg</code> , <code>quick_check_ok</code> , <code>checkpoint_passive</code> , <code>idempotent_schema_init</code>
<code>test_store_edge.py</code>	<code>month_boundary_split</code> , <code>leap_year_feb29</code> , <code>negative_timestamp</code> , <code>nan_value_rejected_at_insert</code> , <code>inf_value</code> , <code>empty_insert_returns_zero</code> , <code>resolve_missing_no_create</code> , <code>get_series_meta_missing</code> , <code>series_ids_empty_metric</code> , <code>list_metrics_unique</code> , <code>no_auto_create_partition</code> , <code>lru_cache_eviction</code> , <code>limit</code> , <code>desc_order</code> , <code>start_gt_end_returns_empty</code> , <code>partitions_for_range_multi_month</code>
<code>test_db.py</code>	<code>result_fetch_helpers</code> , <code>result_iter</code> , <code>connection_commit_rollback_close</code> , <code>execute_error_closes_cursor</code> , <code>executemany_error_path</code> , <code>connect_is_reader_writes_dont_persist</code> , <code>begin_commits_on_success</code> , <code>begin_rolls_back_on_exception</code> , <code>database_execute_persists_writes</code> , <code>executescript</code> , <code>dbapi</code> , <code>missing_parent_dir</code>
<code>test_pragmas.py</code>	<code>wal_enabled</code> , <code>synchronous_normal</code> , <code>busy_timeout</code> , <code>foreign_keys_on</code> , <code>page_size</code> , <code>quick_check_ok</code> , <code>run_optimize</code> , <code>incremental_vacuum</code> , <code>auto_vacuum_incremental</code>
<code>test_persistence.py</code>	<code>data_survives_restart</code> , <code>backup_is_restorable</code> , <code>backup_consistent_while_writing</code>

Query: builder, aggregations, rollup fast path

File	Test functions / coverage
<code>test_query.py</code>	<code>all_rows</code> , <code>time_range</code> , <code>no_metric</code> , <code>order_desc</code> , <code>aggregate_default_avg</code> , <code>aggregate_multiple</code> , <code>whole_window_aggregation</code> , <code>interval_buckets</code> , <code>interval_avg</code> , <code>median</code> , <code>p95_p99</code> , <code>aggregate_functions</code> , <code>bad_aggregation</code> , <code>gap_fill</code> , <code>downsample_buckets</code> , <code>query_stream</code> , <code>missing_series_stream</code> , <code>empty_range</code> , <code>limit</code>
<code>test_agg_edge.py</code>	percentile edge cases: <code>single_value</code> , <code>two_values</code> , <code>exact_index</code> , <code>empty</code> ; empty-window NaN/0 semantics; <code>first/last</code> ; <code>unsupported_func</code> ; <code>parse_interval valid/empty/zero/unsupported</code> ; <code>fill_missing</code> gap buckets; <code>gap_fill_linear thresholds</code> ; <code>downsampling</code>

<code>test_rollup.py</code>	rollup_aggregates, rollup_replace
<code>test_rollup_query.py</code>	query_matches_raw, aggregate_matches_raw, window_within_one_hour, window_crossing_boundary, empty_window, limit, trailing_edge_current_hour, non_expressible_funcs_fall_back, non_aligned_interval_falls_back, no_rollup_data_falls_back, tail_beyond_watermark_falls_back, eligible_uses_rollup, watermark_and_idempotence
<code>test_property.py</code>	Hypothesis invariants: roundtrip_exact, series_identity_stable, aggregate_count_matches, aggregate_sum_matches_python, min_max_bounds, downsample_counts_conserve, downsample_sum_conserve, month_crossing_query_complete, rollup_matches_raw_for_windows

Messaging: transports, protocol, pub/sub

File	Test functions / coverage
<code>test_messaging.py</code>	ingest_roundtrip, invalid_json_counted, publish_delivery, subscription_lifecycle, rep_query, broker_error_handling, runs_steps, invalid_size
<code>test_messaging_edge.py</code>	run_once_before_start_raises, no_handler_returns_invalid_request, invalid_request_reply, handler_error_reply, router_mode_with_dealer, router_missing_payload, invalid_message_counted, skew_rejected, overflowing_timestamp_counted_invalid, recv_error_counted, publish_before_start_raises, step_error_continues, size_zero_raises
<code>test_protocol.py</code>	dumps/loads roundtrip, invalid_json, ingest validation (metric/value/tags/timestamp), to_rows_uses_server_time, skew_rejected_when_configured, query type/metric/start validation, admin cmd validation
<code>test_protocol_fuzz.py</code>	Hypothesis fuzz-lite: parse_ingest/parse_query/parse_admin never crash on arbitrary input; valid shapes accepted or ProtocolError; loads arbitrary bytes; roundtrip; int bounds
<code>test_zmq_edge.py</code>	slow_joiner_loses_early_messages (PUB/SUB semantics), rep_serves_sequential_requests, cleanup_expires_lingering, stats_active_lingering, resubscribe_clears_linger, published_counter, shared_topic_counts_and_decrements

Gateway: REST, WebSocket, rate limiting

File	Test functions / coverage
<code>test_gateway.py</code>	write_single, write_batch, write_invalid_metric, write_bad_body, read_empty, read_data, read_aggregation, read_missing_metric_param, aggregate_multiple, aggregate_bad_func, health, request_id_header, cors_headers, headers_present, headers_on_error_response, stats, ws_subscribe_receives, http_write_publishes_to_ws, ws_connections_push, ws_connections_no_listener_leak, slow_aggregate_does_not_stall_health
<code>test_gateway_edge.py</code>	full 400 catalog: empty/missing metric, non-numeric/bool values, bad tags, invalid timestamps, non-object body, array with invalid item, huge integer value/timestamp → 400 (incl. int64 boundary), batch no-timestamps single transaction, bad aggregation, 422s (order pattern, missing metric, limit over max), far-past timestamp rejection, stats, health
<code>test_ratelimit.py</code>	allows_under_limit, rejects_over_limit, window_resets, different_clients_independent, xff_header_does_not_bypass_limit, hits_table_bounded_within_single_window
<code>test_websocket_edge.py</code>	topic_prefix_filtering, http_write_reaches_subscriber, keepalive_ping_after_idle, message_order_preserved, large_payload_round_trip, malformed_client_data_does_not_break_stream, abrupt_client_disconnect_others_unaffected, oversized_inbound_frame_closes_connection (1009), slow_subscriber_does_not_block_others, rapid_churn_registry_and_listeners_converge, many_concurrent_subscribers, close_1008_when_no_pubsub, cap_closes_overflow_with_1013

Client, service lifecycle, CLI

File	Test functions / coverage
------	---------------------------

<code>test_client.py</code>	<code>write_then_query</code> , <code>write_many</code> , <code>query_aggregation</code> , <code>query_missing_metric_returns_empty</code> , <code>subscribe_streams</code> , <code>close_idempotent</code> , <code>write_then_close_flushes</code> (the consumer-close tests the 2 s linger), <code>query_timeout_recovers</code> , <code>admin_ping/health/stats/unknown/optimize/backup/vacuum</code> , <code>query_error_raises</code> , <code>admin_error_raises</code>
<code>test_service.py</code>	<code>start_shutdown</code> , <code>invalid_config_raises</code> , <code>query_handler</code> , <code>health</code> , <code>starts_with_configured_logging</code> , <code>http_gateway_is_served</code> , <code>rollup_manager_wired</code> , <code>rollup_serves_aggregates</code>
<code>test_service_edge.py</code>	<code>start_failure_cleans_up_and_raises</code> , <code>sink_insert_failure_swallowed</code> , <code>publish_failure_swallowed</code> , <code>query_handler_not_ready</code> , <code>admin_handler_not_ready</code> , <code>negative_limit_invalid_query</code> , <code>admin_backup_failure</code> , <code>admin_vacuum_busy</code> , <code>unknown_admin_command</code>
<code>test_lifecycle_shutdown.py</code>	<code>clean_run_exits_promptly</code> , <code>exits_promptly_after_ws_disconnect</code> , <code>exits_promptly_with_ws_open_at_sigterm</code> , <code>stop_waits_removes_runtime_and_frees_ports</code> , <code>streams_frames_and_cleans_registry</code> , <code>graceful_shutdown_timeout_configured</code> , <code>sigup_terminates_overlay</code> , <code>overlay_dies_when_tmux_session_killed</code>
<code>test_cli.py</code>	<code>help</code> , <code>version</code> , <code>start_help</code> , <code>subcommand_smoke</code> , <code>python_m_sqtseries</code>
<code>test_cli2.py</code>	<code>all_commands_registered</code> , <code>version</code> , <code>health_ok_on_fresh_db</code> , <code>stats_empty</code> , <code>vacuum</code> , <code>optimize</code> , <code>backup</code> , <code>status_not_running</code> , <code>ports_not_running</code> , <code>unit_template</code> , <code>unit_name_const</code> , <code>install_writes_unit_to_home</code> , <code>uninstall_removes_unit</code> , <code>run_rejects_invalid_config</code>
<code>test_cli_edge.py</code>	<code>stop_not_running</code> , <code>stop_stale_pid</code> , <code>status_stale_runtime</code>
<code>test_cli_db.py</code>	<code>--db</code> flag behavior: <code>stats_reads_flagged_db</code> , <code>ports_show_flagged_db_when_not_running</code> , <code>status_flags_conflict</code> , <code>status_silent_when_dbs_match</code> , <code>run_refuses_conflicting_db</code> , <code>run_refuses_same_db_already_served</code> , <code>stop_refuses_wrong_db</code> , <code>stop_reaches_kill_with_matching_db</code> , <code>template_includes_config_when_passed</code> , <code>template_omits_config_when_absent</code> , <code>install_writes_config_file</code>

Config, health, runtime, ports, recovery, backup, maintenance, retention

File	Test functions / coverage
<code>test_config.py</code>	<code>defaults_apply</code> , <code>version_accessible</code> , <code>load_from_toml</code> , <code>load_from_json</code> , <code>load_missing_file_raises</code> , <code>load_default</code> , <code>path_expands_home</code> , <code>env_overrides_defaults</code> , <code>env_overrides_file</code> , <code>env_nested_value</code> , <code>env_leaf_excluded_from_file_override</code> , <code>valid_settings_have_no_errors</code> , <code>invalid_port_range</code> , <code>duplicate_ports</code>
<code>test_config_edge.py</code>	<code>unsupported_extension_raises</code> , <code>json_non_mapping_raises</code> , <code>json_invalid_raises</code> , <code>yaml_requires_pyyaml</code> , <code>batch_size_over_limit</code> , <code>hwm_below_one</code> , <code>pending_max_below_one</code> , <code>invalid_rollup_interval</code> , <code>invalid_retention_ttl</code> , <code>invalid_maintenance_interval</code> , <code>stats_port_collision_reported</code> , <code>stats_port_range_reported</code> , <code>db_path_expanded</code>
<code>test_health.py</code>	<code>collect_health_ok</code> , <code>collect_health_degraded_on_corrupt</code> , <code>is_ready</code>
<code>test_runtime.py</code>	<code>write_read_roundtrip</code> , <code>read_missing</code> , <code>remove</code> , <code>corrupt_file</code>
<code>test_ports.py</code>	<code>alloc_returns_free_port</code> , <code>alloc_skips_used</code> , <code>alloc_deducts_internally</code> , <code>release</code> , <code>exhausted</code> , <code>fixed_port_in_use</code> , <code>offset</code> , <code>auto_detect_never_picks_a_fixed_port</code>
<code>test_recovery.py</code>	<code>quick_check_clean</code> , <code>recover_wal</code> , <code>strict_raises_on_corrupt</code> , <code>non_strict_corrupt_returns_problem_count</code>
<code>test_backup.py</code>	<code>backup_creates_valid_file</code> , <code>backup_dir_created</code> , <code>backup_same_second_raises</code> , <code>latest_returns_none_when_empty</code>
<code>test_backup_and_timeout.py</code>	<code>slow_handler_times_out</code> , <code>fast_handler_with_timeout_succeeds</code> , <code>no_timeout_means_synchronous</code> , <code>service_enforces_configured_timeout</code> , <code>run_once_creates_snapshot</code> , <code>skips_fresh_snapshot</code> , <code>start_stop_lifecycle</code> , <code>is_fresh_helper</code> , <code>backup_interval_validated</code> , <code>disabled_backup_not_validated</code> , <code>service_creates_backups_on_schedule</code> , <code>disabled_backup_no_manager</code> , <code>zero_timeout_runs_handler</code> , <code>admin_conncheck_ids</code>

<code>test_maintenance.py</code>	<code>parse_checkpoint_result</code> , <code>maintenance_runs_analyze_after_interval</code> , <code>checkpoint_detects_blocked_readers</code> , <code>checkpoint_no_wal_no_tracking</code>
<code>test_partition.py</code>	<code>list_empty_plus_current</code> , <code>ensure_creates</code> , <code>rotation_current_month</code> , <code>drop_partition</code> , <code>store_auto_creates_partition</code>
<code>test_retention.py</code>	<code>parses</code> , <code>invalid</code> , <code>retains_recent_only</code> , <code>run_drops</code> , <code>no_drop_when_within_ttl</code> , <code>drop_removes_rollup_rows</code> , <code>drop_rollup_when_no_rollup_table</code> , <code>manager_drops_and_invalidates</code> , <code>manager_start_stop</code>

Connection tracking, async cleanup, concurrency, regressions, docs claims, stress

File	Test functions / coverage
<code>test_connection_tracking.py</code>	<code>register/unregister ws</code> , <code>unregister idempotent</code> , <code>list_connections</code> , <code>zmq_sub_tracking</code> , <code>active_topics</code> , <code>snapshot</code> , <code>event_callbacks</code> , <code>new_connection_id</code> , <code>ws_churn_no_accumulation</code> , <code>remove_listener_idempotent</code> , <code>event_timestamps</code> , <code>unregister_unknown_emits_nothing</code> , <code>zmq_first_seen_round_trip</code> , <code>PubSub start/publish/stop</code> , <code>emit_report</code> , <code>on_registry_event_after_stop</code> , <code>stop_unhooks_registry_listener</code> , <code>xpub_subscription_tracking</code> , <code>admin connections/conncheck/subscribers</code> (empty and with live ws/zmq)
<code>test_async_cleanup.py</code>	<code>stop_cancels_running_tasks</code> , <code>step_after_stop_no_work</code> , <code>managers_stop_idempotent</code> , <code>pubsub_stop_awaits_reader</code> , <code>stats_publisher_stop_awaits_and_drains</code> , <code>subscription_lingering_bounded_without_publishes</code> , <code>shutdown_drains_publish_tasks</code> , <code>shutdown_twice_safe</code> , <code>restart_no_registry_listener_growth</code>
<code>test_concurrency.py</code>	<code>concurrent_writers_no_loss</code> , <code>concurrent_readers</code> , <code>wal_reader_during_write</code>
<code>test_regressions.py</code>	QA-audit guards: <code>sigterm_exits_zero</code> , <code>nan_value_rejected</code> , <code>desc_is_globally_descending</code> , <code>desc_limit_returns_newest</code> , <code>query_after_drop_no_error</code> , <code>failed_insert_leaves_no_orphan_series</code> , <code>late_write_within_skew_is_served_correctly</code> , <code>skew_cutoff_excludes_fresh_hours</code> , <code>invalid_item_rejects_whole_batch</code> , <code>anchor_consistent_asc_desc</code> , <code>empty_env_var_treated_unset</code> , <code>case_variant_env_wins</code> , <code>config_file_with_config_file_key</code> , <code>alloc_skips_reserved</code> , <code>file_expands_home_and_creates_dir</code> , <code>health_degraded_on_corrupt</code>
<code>test_docs_claims.py</code>	verifies docs behaviors: <code>http_read_uses_seconds</code> , <code>client_query_uses_nanoseconds</code> , <code>invalid_query_code</code> , <code>db_path_env_var</code> , <code>config_file_env_var</code> , <code>rate_limit_env_var</code> , <code>health_reply_fields</code> , <code>stats_reply_keys</code> , <code>optimize_reply_field</code> , <code>context_manager</code> , <code>series_ids_query</code> , <code>status_shows_running</code>
<code>test_edge_cases.py</code>	<code>partition_drop_during_query</code> , <code>max_batch_size</code> , <code>timestamp_skew_rejection</code>
<code>test_misc_edge.py</code>	<code>python_m_sqtseries_version</code> , <code>checkpoint_failure_does_not_crash</code> , <code>write_with_timestamp</code> , <code>query_with_limit_and_order</code> , <code>get_series_meta_missing</code>
<code>test_resource_leaks.py</code>	<code>no_resource_warnings_from_db_api</code> , <code>fd_stable_across_operations</code> , <code>lru_caches_clear_on_close</code> , <code>zmq_client_close_no_fd_leak</code> , <code>zmq_contexts_terminate</code>
<code>test_stress.py</code>	<code>sustained_bursts_ok</code> , <code>no_fd_leak</code>
<code>test_stress_concurrent.py</code>	<code>rapid_sub_connect_disconnect</code> , <code>admin_during_churn</code> , <code>stats_immediate_emit</code> , <code>ingest_with_subscribers</code> , <code>conncheck_returns_correctly</code> , <code>stats_report_loop_runs</code>
<code>test_logging.py</code>	<code>console_logger</code> , <code>json_logger</code> , <code>file_output</code> , <code>logger_emits_structured_keywords</code>
<code>test_systemd_edge.py</code>	<code>install_writes_and_enables</code> , <code>install_system_uses_no_user_prefix</code> , <code>uninstall_disables_and_removes</code> , <code>uninstall_system</code> , <code>template_quotes_python_with_spaces</code> , <code>template_defaults</code>

Example runners and browser tests

File	Test functions / coverage
<code>test_examples_run.py</code>	<code>answers_all_four_readme_questions</code> , <code>all_answer_lines_present</code> , <code>producer_and_consumer</code> , <code>ingest_http_then_read</code> , <code>admin_examples</code> , <code>query_examples_every_query_type</code> , <code>tags_examples</code> , <code>run_custom_lifecycle</code>

test_question_examples_advanced.py	advanced_catalog_runs, per_series_questions_need_db, per_series_with_db_works, utc_day_start
test_camera_feed.py	flatten_message, flatten_skips_strings_and_missing, flatten_error_payload, ask_on_empty_db_no_errors, ask_single_point, ask_after_real_pump, ask_q8_recharge_message, random_port_mode, serves_live_dashboard_html
test_camera_overlay_page.py	Playwright (headless Chromium): connects_and_renders_live_values, reconnects_after_server_restart, subscribes_to_sqtseries, db_reconnects_after_sqtseries_restart, connected_clients_live, conn_validate_script

Per-file legend (files with no explicit test list)

test_cli.py / test_cli2.py / test_cli_db.py / test_cli_edge.py cover the CLI;
test_gateway_edge.py / test_websocket_edge.py cover gateway edge cases; test_service*.py cover the daemon lifecycle;
test_zmq_edge.py covers ZMQ semantics — all listed above.

15. docs/ Directory

A hand-written documentation site (HTML + style.css + app.js), not generated. It powers the README links and is itself validated by test_docs_claims.py . Pages:

Page	Covers
index.html	What it is, try it, camera demo, ports
quickstart.html	Install → write → query → subscribe → systemd
configuration.html	Every setting, config file formats, env vars, port rules
ingestion.html	Writing via ZMQ/HTTP/Client, tags, timestamps, the skew guard
queries.html	Ranges, aggregations, intervals, downsampling, gap fill
streaming.html	Live data via SUB, WebSocket, Client; stats events
camera.html	The 17-question camera walkthrough
clients.html	Python, Go, Rust, PHP, Node.js samples
api.html	Embedded API, Client, CLI, wire protocol, admin commands
architecture.html	Engine layout, schema, write path, rollup design, error handling
backup.html	Backup/restore procedure, vacuum
systemd.html	Service hardening, dedicated user setup
examples.html	Real-world scenario walkthroughs
benchmarks.md	Reproducible performance numbers

Supporting assets: style.css , app.js (theme toggle + TOC), favicon.svg .

16. Wire Protocol Reference

Both transports carry the same JSON shapes. Timestamps: *seconds* on the wire (ingest/HTTP and returned rows); ZMQ query *bounds* are nanoseconds; HTTP query bounds are seconds.

16.1 Ingest (ZMQ PUSH → 12501, or POST /api/v1/write)

```
{
  "metric": "cpu.usage",
  "value": 0.72,
  "tags": {
    "host": "web1",
    "timestamp": 1691234567.5
  }
}
```

(fields: metric required non-empty string; value required finite number (no bool); tags optional string-string object; timestamp optional finite number, epoch seconds)

Clock-skew guard: a client timestamp more than `reject_client_timestamp_skew_s` (300 s default) from server time is rejected (ProtocolError / HTTP 400). This protects the rollup's in-order assumption. HTTP accepts a single object or an array (atomic batch).

16.2 Query (ZMQ REQ → 12502, or GET /api/v1/read, /api/v1/aggregate)

```
{
  "type": "query",
  "metric": "cpu.usage",
  "start": 1691234567000000000,
  "end": 1691238167000000000,
  "aggregation": "avg",
  "interval": "1h",
  "limit": 100,
  "order": "asc"
}
{"type": "query", "metric": "cpu.usage", "aggregations": "avg,min,max,p95", "start": ..., "end": ...}
```

reply ok: `{ "status": "ok", "data": [{ "timestamp": 1691234567, "value": 0.72 }, ... ]}`
 or `{ "status": "ok", "data": { "avg": 1.1, "min": 0.2, ... } }` for aggregations
 reply err: `{ "status": "error", "error": { "code": "INVALID_QUERY|INVALID_REQUEST|QUERY_TIMEOUT|INTERNAL_ERROR", "message": "..."} }`

HTTP adds validation: `limit` 1–100000, `order` `^(asc|desc)$`; errors use HTTP status (400/422/504) with the same message style. Aggregations (both transports): `avg, sum, min, max, count, first, last, median, p95, p99`.

16.3 Streaming (ZMQ XPUB → 12503 SUB, or WS /ws/subscribe)

two-part frame: [topic, payload] topic = metric name
 payload: `{ "metric": "cpu.usage", "tags": { ... }, "value": 0.72 }`
 WebSocket: same payload as text frames, plus `{ "type": "ping" }` keepalive when idle ≥30 s

16.4 Admin (ZMQ REQ → 12504) — commands and reply highlights

Command	Request	Reply highlights
<code>ping</code>	<code>{ "cmd": "ping" }</code>	<code>{ "status": "ok", "pong": true }</code>
<code>health</code>	<code>{ "cmd": "health" }</code>	status ok degraded, version, uptime, database bool
<code>stats</code>	<code>{ "cmd": "stats" }</code>	uptime_s, ingested, invalid, ingest_errors, queries, admin_requests, wal_bytes, checkpoints, checkpoint_busy_runs, analyze_runs, retention_runs, partitions_dropped, backup_runs, backups_created, last_backup, published, subscribers, series, metrics
<code>optimize</code>	<code>{ "cmd": "optimize" }</code>	<code>{ "optimized": true }</code>
<code>backup</code>	<code>{ "cmd": "backup" }</code>	<code>{ "path": "... } }</code> or BACKUP_FAILED
<code>vacuum</code>	<code>{ "cmd": "vacuum" }</code>	<code>{ "vacuumed": true }</code> or VACUUM_BUSY (stop the service for full VACUUM)
<code>connections</code>	<code>{ "cmd": "connections" }</code>	data: [{id, kind, peer, topic, connected_at}]

conncheck	<code>{"cmd": "conncheck", "ids": [...]}</code>	present: [ids that are connected]
subscribers	<code>{"cmd": "subscribers"}</code>	zmq_subscribers, subscriptions: [{topic, subscribers, first_seen}]

16.5 Stats events (ZMQ PUB → 12506)

```
topic "conn": {"kind": "ws", "id": ..., "peer": ..., "topic": ..., "connected": true|false,
               "connected_at": ..., "left_at": ...}
topic "sub": {"kind": "zmq", "topic": ..., "subscribers": n, "arrived_at": ..., "left_at": ...,
             "first_seen": ...}
topic "report": {"uptime_s": ..., "ws_connections": n, "zmq_subscribers": n, "active_topics": n}
```

16.6 WS /ws/connections (push-only monitoring feed)

```
{"type": "snapshot", "ws_connections": n, "zmq_subscribers": n, "connections": [...], "subscriptions": [...]}
then one frame per change: {"type": "conn", ...} / {"type": "sub", ...}
```

17. End-to-End Data Flows

17.1 Write path (ZMQ)

```

producer (PUSH) —frame→ Ingress.run_once → _handle:
  orjson.loads → parse_ingest (+skew guard) → _sink("cpu.usage", tags, 0.72, ts_ns)
  ↓
  on_publish(metric, {metric, tags, value}) → PubSub.publish → [topic, json] on XPUB 12503
  ↓
  WS /ws/subscribe clients via SUB socket
  
```

StorageEngine.insert_many (ONE transaction: resolve series, create partition if needed, executemany into measurements_YYYY_MM)

17.2 Write path (HTTP)

POST /api/v1/write → validate the whole body (400 on any bad row) → one-transaction `insert_many` (batch) → best-effort `_broadcast` to PubSub → `{"status": "ok", "written": n}`. Timestamp-less rows in a batch get distinct server-ns values so the clustered PK can't collide.

17.3 Query path

```

Client.query / HTTP /read → TimeSeriesDB.query
├── _rollup_eligible? — no → raw fetch (query_time_range per partition)
│   └── downsample / whole-window aggregate / gap-fill in Python
└── yes → _rollup_window_partials: raw LEFT edge hour + rollup MIDDLE hours
    (query_rollup_partial) + raw RIGHT tail → _merge_partials → _finalize_buckets
    (or whole-window _value anchored at first_sample_ts)
    └── [{"timestamp": ts_s, "value": v}, ...]
  
```

17.4 Rollup background job

```

every rollup.interval (5m): rollup_new_hours(db, skew_s=300)
  completed = floor(now - skew) to the hour
  watermark = rollup_meta.last_hour
  for each partition intersecting [watermark, completed):
    GROUP BY series_id, hour → INSERT OR REPLACE INTO rollup_hourly
  advance rollup_meta.last_hour = completed
  
```

17.5 Retention background job

```

every check_interval (1h): RetentionPolicy.run()
  horizon = now - ttl (30d)
  drop partitions whose LAST day < horizon (DROP TABLE + DELETE their rollup_hourly hours)
  
```

17.6 Lifecycle

```

sqtseries run → conflict check vs runtime.json → migrate + quick_check + recover_wal
  → start managers → bind ZMQ sockets → start uvicorn → ANALYZE → write runtime.json
  → wait for SIGINT/SIGTERM
  → stop managers → stop sockets → uvicorn.should_exit → drain publishes → optimize
  → TRUNCATE checkpoint → dispose engine → remove runtime.json
  
```

18. Glossary

Term	Meaning
Measurement	One data point: metric + value (+ tags + timestamp).
Metric	Named series identifier, e.g. <code>cpu.usage</code> . Queries address whole metrics over the wire.
Series	The unique <code>(metric, tags)</code> combination; owned by a <code>series_id</code> .
Tags	String → string dimensions on a measurement; each distinct set creates a new series.
Partition	Monthly table <code>measurements_YYYY_MM</code> (WITHOUT ROWID, PK <code>series_id+timestamp_ns</code>); created lazily, dropped by retention.
Watermark	<code>rollup_meta.last_hour</code> — the next hour the rollup still has to aggregate.
Skew guard	<code>reject_client_timestamp_skew_s</code> — rejects client timestamps too far from server time; keeps the rollup strictly in order.
TTL	Retention horizon ("30d") after which whole partitions are dropped.
WAL	SQLite Write-Ahead Log journal — crash-atomic writes; checkpointed by CheckpointManager and on shutdown.
HWM	ZeroMQ high-water mark — queue limit before backpressure/drop.
XPUB	PUB socket that also receives subscription events — used to track ZMQ subscribers exactly.
XPUB_VERBOSE	libzmq option emitting a subscription event for EVERY join/leave (not only topic-trie transitions).
runtime.json	File next to the database recording pid, active ports, db path, start time; written on start, removed on clean shutdown.
fast path	The rollup-accelerated query path; falls back to raw scanning when ineligible.
embedded API	Direct in-process Python API (StorageEngine/TimeSeriesDB on the DB file) — required for per-series reads the wire protocol can't express.